

Desarrollador: Depuración con Trazas en Cliente



© 2013 Meta4 Spain, S.A. Se reservan todos los derechos.

AVISO: Este manual está protegido por la legislación referente a propiedad intelectual e industrial y por tratados internacionales. La utilización permitida de esta documentación queda limitada a su uso en conexión con el producto, y todo uso no autorizado será perseguido de acuerdo con la legislación aplicable. Se prohíbe su copia, modificación, reproducción o distribución sin permiso del titular.

Meta4 PeopleNet © 1999 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 KnowNet © 1996 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 e-mind © 2001 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 PeopleNet Ksystem © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 t.innova © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4[®], Meta4Mind[®], Meta4 PeopleNet[®], Meta4 KnowNet[®], Meta4 e-mind[®], Meta4 PeopleNet Ksystem[®] y Meta4 t.innova[®] son marcas registradas propiedad de Meta4Spain, S.A.

Otros nombres de compañías, productos o servicios son marcas registradas o nombres comerciales de sus respectivos propietarios.

Meta4 Spain, S.A.
Centro Europa Empresarial
Edificio Roma
C/ Rozabella, 8
Ctra. de La Coruña, km 24,200
28290 Las Rozas, Madrid
ESPAÑA
<http://www.meta4.com>

Fecha de creación: 30 de marzo de 2005.

Fecha de la última publicación: diciembre de 2013.

Tabla de contenidos

1 Introducción a las trazas

■ Acerca de este capítulo	7
■ Sistema de traza y registro de eventos	8
Clientes y el servidor de aplicaciones	8
Clientes y los servicios Web	8
■ Tipos de trazas	9
Trazas en cliente Windows de desarrollo y en servidor	10
Ldbinsp.txt	10
Archivos de log o m4logfile.log	10
Trazas del motor de informes	11
Trazas de cachés	11
Trazas de tramos: slices.log	11
M4oltp.log	11
M4metadata.log	12
Logon.log y Params.log	12
ChannelManager.txt	12
Autocache.txt	13
Planificador de tareas	13
Trazas en el cliente Windows de desarrollo	13
CSTrace.txt	13
Trazas en el Servidor de aplicaciones	13
M4benchmark.log	14
Trazas de la máquina virtual (PDUs)	14
M4xml.log	14
M4eventlog.log	14
M4auditorytrace.log	15
Trazas en el Web	15
Servicios Web	15
Cliente Web	15
Cliente Rich Web	16
Bootstrap: archivo m4rwboot.log	16
Systray: archivo m4systray.log	16

M4loader: archivo m4loader.log	16
M4uninstaller: archivo m4uninstaller.log	16
■ Configuración común a múltiples trazas.	17
Ubicación de las trazas	17
Activación de las trazas	17
Tiempo de validez de las trazas.	18

2 Archivos de trazas en servidores de aplicación y en clientes

■ Acerca de este capítulo	19
■ Archivos de traza comunes al servidor y el cliente.	20
Tipos de error que se guardan	20
Identificación de errores	20
■ El archivo ldbinsp.txt.	21
Configuración del fichero de trazas ldbinsp.txt	21
Definición de la traza de severidad de errores	22
Definición de la traza de BDL de severidad de errores	22
Definición del nivel de detalle de la traza de BDL	23
Combinación de la definición de la traza de BDL y sus niveles	23
Almacenamiento de los archivos de traza	25
Cambio de tamaño	25
Cambio de número de copias de seguridad	26
Cambio de severidad de errores	26
Cambio de nivel de severidad de errores	27
Cambio del formato	28
■ Archivos de log	29
Configuración de los archivos de log	29
Ubicación	29
Sólo para servidores de aplicación	30
Formato del archivo logbase.ini	30
Parámetros globales	30
Dispositivos de salida	31
PRIVATE_MESSAGE_QUEUE	32
PUBLIC_MESSAGE_QUEUE	32
PUBLIC_LOG_FILE	32
Grupos de dispositivos de salida	32
Patrones de salida [outputPattern]	33
Módulos de aplicación [applicationModule]	34
Formato de los archivos logsys.ini y logsysserver.ini	34
Cambio de severidad de errores	35

Establecimiento del nombre del dispositivo de salida	35
Definición del tipo de dispositivo	35
Cambio del formato de salida	36
Elección del idioma del archivo de traza	37
Códigos de error en los archivos de log	37
■ El archivo M4oltp.log	39
■ El archivo M4metadata.log	40
■ Los archivos logon.log y params.log	41
■ Trazas del motor de informes	42
Configuración de las trazas de informes	42
Filtro del nivel de traza	42
■ Trazas de tramos	44
■ Trazas de Meta4Objects del tipo MT	46
■ El archivo ChannelManager.txt	47
■ Trazas de depuración: autocache.log	48
■ Configuración de las cachés	50
Cachés de Metadatos	51
Cachés de presentaciones	51
Cachés de datos	52
Cachés de seguridad	52
Formato de los archivos	52
■ Trazas del planificador de tareas	56

3 Trazas en modo desarrollo

■ Acerca de este capítulo	59
■ El archivo CSTrace.txt	60
■ Trazas de presentaciones	62
Configuración del archivo de trazas	63
Niveles de trazas	63
■ Trazas de transferencia de datos entre Meta4Objects	64
■ Trazas del cliente ligero	65
Trazas de compilación e instanciación de páginas JSP	65
Trazas necesarias en la depuración de páginas JSP	65
Trazas generados mediante las funciones Log de LN4	65

4 Apéndice: Trazas de cliente servidor

■ Introducción	67
----------------------	----

■ Ejemplos de Trazas	68
Ejemplo de archivo a nivel 1	68
Ejemplo de archivo a nivel 2	69

5 Apéndice: Volcado de un rango de errores a un nuevo archivo

■ Introducción	71
Modificaciones en el archivo logbase.ini	71
Modificaciones en el archivo logsys.ini	74

6 Apéndice: Depuración en informes

■ Introducción	75
■ Activación y desactivación de trazas de informe	76
■ Definición del nivel de depuración	77

7 Apéndice: Modelo de datos del planificador de tareas y Volcado de trazas

■ Introducción	79
■ Modelo de datos del planificador de tareas	80
Definición de los datos suministrados por el modelo	80
Datos de la planificación	80
Datos de las ejecuciones	80
Configuración de la instalación	80
Datos de sistema	81
■ Volcado de trazas	81

Introducción a las trazas

1

Acerca de este capítulo

En este capítulo se pretende describir a grandes rasgos las trazas disponibles para depurar una aplicación Meta4, como Meta4 PeopleNet Kystem desde el cliente, tanto como los servidores de aplicación y de Web.

De este modo cualquiera tendrá una idea básica de las trazas que se puede utilizar en la detección de problemas que pueden estar ubicados en diferentes partes de la plataforma Meta4 del producto correspondiente.

Sistema de traza y registro de eventos

Clientes y el servidor de aplicaciones

Se encarga de almacenar toda la información relativa a las incidencias ocurridas en el servidor de aplicaciones y las otras en los clientes durante el proceso de trabajo.

El servidor de aplicaciones cuenta con varios mecanismos para informar tanto de su estado como del de las peticiones que está ejecutando.

Los clientes reciben los errores que se generan en el servidor de aplicaciones y el servidor de aplicaciones guarda un archivo en el que se enumeran los errores producidos en el mismo y los enviados a los clientes. El sistema genérico de traza es parametrizable y flexible para permitir, entre otras cosas, filtrar la gravedad y el formato de los errores que se registran.

Por otra parte, la base de datos lógica dispone de su propio archivo de trazas, en el que se con una parametrización adecuada se pueden obtener las sentencias SQL que el servidor de aplicaciones envió a la base de datos con el fin de depurar errores complicados en la conclusión de transacciones de usuarios.

El monitor del servidor de aplicaciones además permite recoger los eventos que se producen en el servidor de aplicaciones, que son todas aquellas incidencias que tienen lugar en el Servidor de aplicaciones mientras que éste se encuentra realizando las peticiones recibidas de los sistemas clientes.

La gran mayoría de estas incidencias se envían al administrador del sistema, que será quien tome las medidas oportunas, pero también se envían a los sistemas cliente los distintos mensajes emergentes como incidencias de proceso.

Clientes y los servicios Web

Además en la arquitectura Meta4 que también incluye a los servicios Web, también existen trazas para la configuración de clientes Web que accederán a un contenedor Web que a su vez tiene los servicios Web de la funcionalidad del producto Meta4. En este caso existen trazas tanto para el cliente Web como para los servicios Web.

Tipos de trazas

Las trazas disponibles se pueden organizar según el tipo de usuario que las necesite en la siguiente manera:

- Las trazas para desarrolladores en los clientes Windows de desarrollo.
- Las trazas para desarrolladores en los clientes ligeros.
- Las trazas para administradores en los servidores de aplicaciones.
- Las trazas para administradores en los servidores Web.
- Las trazas de instalación del cliente Rich Web.

Las trazas disponibles no significan que por defecto se generan, hay que activarlas antes. La manera de activarlas se explicará en los capítulos siguientes.

Tabla 1. Tipos de trazas disponibles en diferentes máquinas una vez

Tipo Traza	Cliente Windows de desarrollo	Servidor de aplicaciones	Cliente Rich Web
Ldbinsp.txt	✓	✓	
Archivos log o m4logfile.log	✓	✓	✓
Informes	✓	✓	✓
Slices.log	✓	✓	✓
M4oltp.log	✓	✓	
M4metadata.log	✓	✓	
Logon.log y Params.log	✓	✓	✓
Archivos Meta4Object MT	✓	✓	✓
ChannelManager.txt	✓	✓	✓
Cache Dump	✓	✓	✓
Planificador de tareas	✓	✓	
Autocache.log	✓	✓	✓
CSTrace.txt	✓		✓
M4benchmark.log		✓	
Maquina Virtual - PDU		✓	
M4xml.log		✓	
M4eventlog.log		✓	
M4auditorytrace.log		✓	
m4rwboot.log			✓

Tabla 1. Tipos de trazas disponibles en diferentes máquinas una vez

Tipo Traza	Cliente Windows de desarrollo	Servidor de aplicaciones	Cliente Rich Web
m4systray.log			✓
m4loader.log			✓
m4uninstaller.log			✓

A continuación se describirán las trazas disponibles según su ubicación en cliente o en servidor. En cada apartado se describe la finalidad y las indicaciones de cada traza y se adjunta la ruta por defecto de estas trazas y si la activación defecto es la recomendada.

Trazas en cliente Windows de desarrollo y en servidor

Estas trazas están disponibles tanto para un desarrollador desde su máquina cliente de desarrollo como para un administrador desde el servidor de aplicaciones.

Ldbinsp.txt

Contiene detalles de la ejecución y tiempo consumido en cada sentencia de base de datos.

Traza de la carga de datos y/o metadatos por la aplicación. Tiempos de las sentencias de base de datos.

Ubicación por defecto:

%M4_SERVER_CONFIGURATION_ROOT%\temp\m4debug

%M4_CLIENT_CONFIGURATION_ROOT%\ M4temp

Activación por defecto recomendada: NO, salvo en el caso de cliente Windows de desarrollo con el nivel 1 de trazabilidad en SYSTEMDEBUGDETAILLEVEL. Consulte el apartado El archivo ldbinsp.txt en el capítulo *Archivos de trazas en servidores de aplicación y en clientes* para más información.

Archivos de log o m4logfile.log

Registra los errores, avisos e información de depuración producidos por la aplicación.

Puede consultar más información sobre su ubicación y configuración en el apartado "Configuración común a múltiples trazas" de este capítulo.

Esta traza se encuentra siempre activada.

Trazas del motor de informes

Trazan las operaciones realizadas por el motor de informes según el filtro indicado.

Ubicación por defecto:

%M4_SERVER_CONFIGURATION_ROOT%\temp\m4reports

%M4_CLIENT_CONFIGURATION_ROOT%\ M4temp\M4Reports

Activación por defecto recomendada: NO

Trazas de cachés

Registran la actividad de lectura y escritura en los archivos de caché.

Ubicación por defecto:

%M4_SERVER_CONFIGURATION_ROOT%\temp\m4debug

%M4_CLIENT_CONFIGURATION_ROOT%\ M4temp

Activación por defecto recomendada: NO

Trazas de tramos: slices.log

Las trazas de tramos se describen en el archivo slices.log y indican como se ha tramado un concepto o varios conceptos determinados y muestra cada tramo generado.

Ubicación por defecto:

%M4_CLIENT_CONFIGURATION_ROOT%\M4debug

Activación por defecto recomendada: NO

M4oltp.log

Meta4Object, nodo y método de una transacción OLTP.

Registro de las transacciones OLTP desencadenadas por cada objeto. Se combina con el m4benchmark.log.

Puede consultar más información sobre su ubicación y configuración en el

apartado "Configuración común a múltiples trazas" de este capítulo.

Activación por defecto recomendada: NO

M4metadata.log

Objeto correspondiente a las transacciones de metadatos.

Traza de las transacciones de metadatos. Se combina con el m4benchmark.log.

Puede consultar más información sobre su ubicación y configuración en el apartado "Configuración común a múltiples trazas" de este capítulo.

Activación por defecto recomendada: NO

Logon.log y Params.log

Operaciones y validaciones realizadas durante el proceso de conexión.

Registro de incidencias en el acceso a la aplicación y comprobación de la correcta aplicación de los parámetros a las sesiones de los usuarios.

Ubicación por defecto

%M4_SERVER_CONFIGURATION_ROOT%\temp

%M4_CLIENT_CONFIGURATION_ROOT%\temp

Activación por defecto recomendada: NO

ChannelManager.txt

Operaciones y validaciones realizadas durante el proceso de conexión.

Registro concatenado de los Meta4Objects que se van llamando en una sesión.

Ubicación por defecto

%M4_SERVER_CONFIGURATION_ROOT%\temp

%M4_CLIENT_CONFIGURATION_ROOT%\temp

Activación por defecto recomendada: NO

Autocache.txt

Trazas de los Meta4Objects del tipo Delta que se han guardado en el caché delta.

Ubicación por defecto

%M4_SERVER_CONFIGURATION_ROOT%\temp\m4debug

%M4_CLIENT_CONFIGURATION_ROOT%\ temp\m4debug

Activación por defecto: NO

Planificador de tareas

Las trazas del planificador de tareas están en la base de datos. Se puede consultarles desde cualquier máquina.

Trazas en el cliente Windows de desarrollo

Estas trazas están disponibles únicamente para un desarrollador desde su máquina cliente.

CSTrace.txt

Volumen de información y tiempo consumido en las llamadas desde el cliente al servidor y viceversa.

Verificación del tráfico de datos entre cliente y servidor. Comprobación de la correcta configuración C/S de la aplicación.

Ubicación por defecto:

%M4_CLIENT_CONFIGURATION_ROOT%\M4temp

Activación por defecto recomendada: NO

Trazas en el Servidor de aplicaciones

Estas trazas están disponibles únicamente para un administrador desde un servidor de aplicaciones.

M4benchmark.log

Tiempo consumido en cada módulo del servidor de aplicaciones.

Traza de la actividad de los threads (ejecutores) de un servidor y los tiempos empleados en cada transacción.

Puede consultar más información sobre su ubicación y configuración en el apartado "Configuración común a múltiples trazas" de este capítulo.

Activación por defecto recomendada: SI

Trazas de la máquina virtual (PDUs)

Almacena las unidades de proceso de datos.

Traza de la secuencia de peticiones entre servidor y cliente.

Ubicación por defecto:

%M4_SERVER_CONFIGURATION_ROOT%\temp

Activación por defecto recomendada: NO

Sólo se debe usar de forma puntual, en concreto si se lo pide Soporte Meta4 para tareas diagnósticas ya que consume mucha memoria.

M4xml.log

Objeto correspondiente a las transacciones xml.

Traza de las transacciones xml. Se combina con el m4benchmark.log.

Puede consultar más información sobre su ubicación y configuración en el apartado "Configuración común a múltiples trazas" de este capítulo.

Activación por defecto recomendada: NO

M4eventlog.log

Todos los eventos generados durante el proceso de conexión.

Puede consultar más información sobre su ubicación y configuración en el apartado "Configuración común a múltiples trazas" de este capítulo.

Activación por defecto: SI, siempre está activada.

M4auditorytrace.log

Operaciones y validaciones realizadas durante el proceso de conexión.

Registro de sesiones establecidas y cerradas por usuarios contra las tablas de la aplicación durante la vida del servidor.

Puede consultar más información sobre su ubicación y configuración en el apartado "Configuración común a múltiples trazas" de este capítulo.

Activación por defecto: SI.

Trazas en el Web

Servicios Web

Las trazas disponibles en el servidor Web sólo se mencionarán por encima en este manual, para mayor información consulte el manual de *Administración de servicios Web*.

Por defecto se obtienen varios archivos de traza, de los cuales los más importantes son:

- **webservices.log**: contiene las trazas del cliente ligero.
- **connect.log**: contiene un archivo con la comunicación XML entre el servidor Web y el Servidor de aplicaciones.
- **jsp.log**: contiene mensajes obtenidos de las páginas.
- **sessions.log**: registra los usuarios conectados.

Cliente Web

Las trazas disponibles en el servidor Web sólo se mencionarán por encima en este manual, para mayor información consulte el manual de *Administración de servicios Web* y el manual *HTML Development Framework*.

Por defecto se obtienen varios archivos de traza, de los cuales los más importantes son:

- **webservices.log**: contiene las trazas del cliente ligero.
- **jsp.log**: contiene mensajes obtenidos de las páginas.

- logs generados de instanciación de clases y de compilación de páginas JSP.

Cliente Rich Web

Los archivos de traza relativos a la instalación del cliente Rich Web son los siguientes:

Bootstrap: archivo m4rwboot.log

- Ruta: %temp%
- Configuración: no. Siempre se genera.
- Tipo de información: actividad del bootstrap. Registra todas las peticiones que recibe, las acciones intermedias y el resultado final. Instalación de paquetes, descarga de ficheros, inyección de recursos, auto-actualización del boot, registro manual de componentes.

Systray: archivo m4systray.log

- Ruta: %temp%
- Configuración: no. Siempre se genera.
- Tipo de información: monitorización del proceso Mind.exe que lo arrancó y envío de petición de desconexión al servidor, cuando detecta que el proceso ha finalizado.

M4loader: archivo m4loader.log

- Ruta: %temp%
- Configuración: no. Siempre se genera.
- Tipo de información: funciones y parámetros ejecutados desde este componente, así como el posible retorno de errores en caso de fallo.

M4uninstaller: archivo m4uninstaller.log

- Ruta: %temp%
- Configuración: no. Siempre se genera.

- Tipo de información: listado de módulos RichWeb instalados (nombre del producto, código del producto, estado del producto y ruta de instalación). En el caso de desinstalar alguno de ellos, se guardará el resultado de la operación.

Configuración común a múltiples trazas

Para que la gestión de los ficheros de traza durante la vida de la aplicación o en ejecuciones posteriores sea más sencilla, uniforme y coherente, las siguientes trazas comparten tanto su ubicación como su configuración:

- M4benchmark.log
- M4oltp.log,
- M4metadata.log
- M4xml.log
- M4logfile.log
- M4sm_trace.log
- M4wf_trace.log
- M4dmstrace.log
- M4eventlog.log
- M4auditorytrace.log

Ubicación de las trazas

Su ubicación se especifica en la entrada **DebugDir** del archivo regmeta4.xml de registro de aplicación. Por ejemplo, para el servidor de aplicaciones:

```
[HKEY_LOCAL_MACHINE\SOFTWARE\META4\MIND\3.X\Build\M4R&D\APPServer\CVM
\LOG]
"DebugDir"="c:\m4serversite-local\m4appserver-config\temp\m4debug"
```

Activación de las trazas

Para activar estas trazas, basta definir `SystemDebugLevel = 1` (el valor por defecto es cero) en la sección `CVM.LOG` de la configuración de la aplicación, que se encuentra en:

- Cliente Windows: en el nodo `CVM.LOG` del fichero de configuración regmeta4.xml. Localice la cadena **SystemDebugLevel** en el archivo.

- Servidor: en la entrada de registro **SystemDebugLevel** que se encuentra bajo la rama *CVM.LOG*. En plataformas Windows en el registro del sistema, en plataformas UNIX en el fichero regmeta4.reg.

```
HKEY_LOCAL_MACHINE\SOFTWARE\Meta4\Mind\3.X\Build\Nombre de la
instancia\APPServer\CVM\LOG
```

Es posible modificar el nivel de traza desde el monitor del servidor de aplicaciones mediante la propiedad **Set Virtual Machine Debug Level** de la ruta **Traces/VM Traces**.

Tiempo de validez de las trazas

La propiedad **MaxDebugPeriod** permite configurar el tiempo de validez de la información de los archivos de traza, que determina la obsolescencia de los backups de los ficheros de traza. La unidad empleada para definir el periodo de validez son las horas y su valor por defecta es 24.

Este parámetro **MaxDebugPeriod** se modifica en la sección *CVM.LOG* de la configuración de la aplicación, que se encuentra en:

- Cliente Windows: en el nodo *CVM.LOG* del fichero de configuración regmeta4.xml. Localice la cadena **MaxDebugPeriod** en el archivo.
- Servidor: en la entrada de registro **MaxDebugPeriod** que se encuentra bajo la rama *CVM.LOG*. En plataformas Windows en el registro del sistema, en plataformas UNIX en el fichero regmeta4.reg.

```
HKEY_LOCAL_MACHINE\SOFTWARE\Meta4\Mind\3.X\Build\M4R&D\APPServer\CVM\
LOG
```

Cuando se vuelva a iniciar el Servidor de aplicaciones o el cliente, se utilizarán los nuevos valores introducidos.

Archivos de trazas en servidores de aplicación y en clientes

2

Acerca de este capítulo

En este capítulo se describen las trazas que están disponibles tanto para administradores en los servidores de aplicación como para los desarrolladores en sus máquinas clientes Windows. Además estas trazas son las que probablemente más se utilizarán para depurar problemas encontrados.

Archivos de traza comunes al servidor y el cliente

El archivo **logbase.ini** es el encargado de parametrizar el sistema de trazas por defecto, que posteriormente es modificado por el establecido en **logsysserver.ini** en el servidor y en **logsys.ini** en el cliente. El archivo generado, **mlogfile.log** reflejará la configuración establecida en los archivos .ini mencionados.

Tipos de error que se guardan

Se distinguen dos grandes tipos de error:

- Tecnológicos: todos aquellos errores de bajo nivel producidos en el funcionamiento interno del servidor, en el cliente o en sus comunicaciones.
- Funcionales: aquellos errores ocurridos durante la realización de las tareas solicitadas por los clientes, que se encuentran en bases de datos.

Identificación de errores

Para identificar claramente los errores producidos en el **m4logfile.log**, se utiliza un formato propio de Meta4 que se estructura en los siguientes apartados:

- Tipo: clase de error.
- Fecha/Hora: momento en el que se produjo la incidencia.
- Código: identifica el mensaje en formato decimal y hexadecimal.
- Mensaje: texto explicativo del error tratado.

Se puede configurar para utilizar el mismo archivo m4logfile.log empleando su formato.

El archivo ldbinsp.txt

El Inspector de la base de datos lógica lleva un seguimiento de todas las operaciones de base de datos solicitadas a la base de datos lógica (BDL) y genera un archivo de traza **ldbinsp.txt** donde se pueden revisar las causas de cualquier incidente ocurrido.

Las aplicaciones Meta4 disponen de una funcionalidad que permite registrar en archivos de texto la actividad realizada por la base de datos lógica. En ellos se registran los accesos a la base de datos y la información que recibe la aplicación. Los archivos donde se vuelcan las trazas de la base de datos lógica, **ldbinsp.txt**, resultan en ocasiones complicados de analizar por la gran cantidad de información que se muestra. Es posible que gran parte de dicha información sea innecesaria en determinados casos.

Configuración del fichero de trazas ldbinsp.txt

Tras la instalación, este sistema de traza está desactivado por defecto. Para activarlo deben modificarse parámetros de la configuración. Estos parámetros se modifican en la sección *CVM.LOG* de la configuración de la aplicación, que se encuentra en:

- **Cliente Windows:** En el nodo *CVM.LOG* del fichero de configuración *regmeta4.xml*
- **Servidor:** En la entrada de registro que se encuentra bajo la rama *CVM.LOG*. En plataformas Windows en el registro del sistema, en plataformas UNIX en el fichero *regmeta4.reg*.

HKEY_LOCAL_MACHINE\SOFTWARE\Meta4\Mind\3.X\Build\Nombre de la instancia\APPServer\CVM\LOG

En esta ruta existe una serie de parámetros, que se describen a continuación:

- **DebugDir:** ruta absoluta donde se generan los archivos de traza. Por defecto se establece en el directorio temporal de instalación.
- **MaxLDBInspSize:** valor por defecto 10. Es el tamaño total en Megabytes de los archivos de traza. Por defecto se generan dos archivos, *ldbinsp0_1.txt* y *ldbinsp0_2.txt*, los cuales se van rellenando alternativamente hasta alcanzar la mitad del tamaño máximo definido. En ese momento la traza comienza a escribir en el otro archivo. De esta manera se van sobrescribiendo alternativamente. Como consecuencia de esto sólo se tendrá registrada la actividad de base de datos de las últimas acciones con un máximo de tamaño 10 MB.

MultipleFilesTrace: valor por defecto 0. Únicamente válido para el servidor de aplicaciones, determina si se genera un archivo por cada conexión lógica del servidor, o bien se graba toda la actividad a un mismo archivo. Tiene dos valores posibles: 0 y 1. En caso de tomar valor 1 separa la información en distintos archivos, un par de ellos por conexión. El número de archivos depende del número de conexiones lógicas definidas en la entrada `MaxNumConn` de la ruta `CVM.LDB` de la configuración.

Por defecto el valor se establece a 50, con lo que habrá una serie de 50 parejas archivos `ldbinsp1_1.txt` y `ldbinsp1_2.txt`, `ldbinsp2_1.txt` y `ldbinsp2_2.txt`, etc. En ellos aparecerá registrada individualmente la actividad de cada conexión. Es muy posible que, si el servidor de aplicaciones no lo necesita, no llegue a registrar actividad en todas las conexiones posibles por lo que no necesariamente se emplearán todos aunque aparecerán creados.

- **ShowData:** valor por defecto 0. Tiene dos valores posibles: 0 y 1. Si el valor es 0, las trazas ocultan los datos resultantes de las sentencias de base de datos reemplazando por asteriscos (*) los caracteres.
- **SystemDebugEnabled:** valor por defecto 0. Permite activar o desactivar trazas en varios niveles. Valores posibles 0, 1, 2 y 3. Consulte el apartado Definición de la traza de severidad de errores para mayor información.
- **SystemDebugDetailLevel:** valor por defecto 0. Permite filtrar determinada información desechando para cada nivel de traza aquella que sea superflua para ciertos casos. Valores posibles 0, 1, 2, 3 y 4. Consulte el apartado Definición de la traza de severidad de errores para mayor información.

Si no aparece `SystemDebugDetailLevel` en el registro, agréguelo a mano.

Definición de la traza de severidad de errores

El archivo donde se vuelcan las trazas de la base de datos lógica, **ldbinsp.txt**, a veces resulta complicado de analizar por la gran cantidad de información que se muestra. Se puede filtrar la información a mostrar, configurando una traza en función del nivel de detalle que se requiere.

A tal fin, se configura las propiedades del archivo `regmeta4.reg`, **SystemDebugEnabled** para activar o desactivar trazas, y luego **SystemDebugDetailLevel**.

Definición de la traza de BDL de severidad de errores

La propiedad **SystemDebugEnabled** permite indicar si se habilita o inhabilita el volcado de trazas.

Existen cuatro niveles de severidad de errores que se especifican en la propiedad **SystemDebugEnable** del archivo regmeta4.reg de cada configuración de servidor. Esta propiedad tiene alguno de los siguientes valores:

- 0, sin traza
- 1, traza de datos (sentencias de usuario)
- 2, traza de datos y metadatos (sentencias de usuario y sistema)
- 3, traza de datos, metadatos y funciones de BDL (traza de mayor nivel); esta traza no se suele usar, normalmente se reserva para las tareas diagnosticas de Soporte Meta4.

Definición del nivel de detalle de la traza de BDL

En caso de habilitar el volcado de trazas, es decir, **SystemDebugEnable** contiene un valor que no sea 0, la propiedad **SystemDebugDetailLevel** permite indicar la manera de filtrar la información que se vuelque. Esta propiedad es opcional, es decir, en caso de no existir la entrada de registro correspondiente o si su valor es 0, se interpretará que no hay detalle y por lo tanto no influirá en el volcado de trazas que había implementado hasta el momento.

La propiedad **SystemDebugDetailLevel** está en el mismo nivel que **SystemDebugEnable**:

```
"SystemDebugEnable"="1"  
"SystemDebugDetailLevel"="1"
```

La propiedad **SystemDebugDetailLevel** puede tomar diferentes valores, dependiendo del nivel de traza que se desee. Los niveles de traza son:

- 0, no se aplica detalle.
- 1, se muestran detalles relacionados con la ejecución y tiempos.
- 2, se muestran todos los detalles (por ejemplo, ejecución, datos, sentencias de PREPARE, FETCH etc).
- 3, se muestran detalles relacionados con la ejecución, tiempos y datos.
- 4, se muestran detalles relacionados con ejecuciones, tiempos, datos e información de filtros de seguridad.

Los mensajes de errores y advertencias se seguirán mostrando.

Combinación de la definición de la traza de BDL y sus niveles

Al parametrizar las propiedades **SystemDebugEnable** y **SystemDebugDetailLevel**, se puede definir la traza de BDL utilizando diferentes tipos de filtrados a base de los diferentes valores que se asigna a

cada propiedad. La combinación de valores disponibles se representan en la siguiente tabla:

SystemDebugEnabled	SystemDebugDetailLevel	Contenido
0	0	Sin traza.
1	0	Traza de datos.
2	0	Traza datos y metadatos.
3	0	Traza de mayor nivel posible
0	< > 0	Sin traza.
1	< > 0	Traza de datos con el filtrado especificado.
2	< > 0	Traza de datos y metadatos con el filtrado especificado.
3	< > 0	Traza de mayor nivel posible con el filtrado especificado

Los valores de filtrado posibles determinan el detalle que puede obtenerse. Cabe destacar que una traza con el nivel de detalle máximo u sin filtro es más extensa y puede ser más difícil de interpretar. Los posibles tipos de filtro se listan a continuación.

SystemDebugDetailLevel	Tipo de filtro
0	■ No se aplica filtro. Se vuelcan todos los datos.
1	■ Se muestran detalles relacionados con la ejecución y tiempos, pero no datos.
2	● Todos los detalles (ejecución, datos, prepares, bindados...). No filtros de seguridad. ● No significa que sea equivalente a SystemDebugEnabled = 1 y SystemDebugDetailLevel = 0. La información que se vuelca con los valores anteriores es mucho más extensa que con SystemDebugEnabled = 1 y SystemDebugDetailLevel = 2.
3	■ Se muestran detalles relacionados con ejecuciones, tiempos y datos. Como el nivel 1 pero con datos. No muestra prepares, bind ni filtros de seguridad.
4	■ Se muestran detalles relacionados con ejecuciones, tiempos, datos e información de filtros de seguridad, pero no datos.

Si el valor es menor que cero y mayor que cuatro se tomará valor cero.

Los valores que tomarán estas variables serán los establecidos en el registro en el momento arranque de los puestos cliente de desarrollo y de los servidores de aplicaciones, no variando de forma dinámica si cambiase dicho valor durante la sesión. Sin embargo, para los servidores de aplicaciones es posible los parámetros en tiempo de ejecución desde la herramienta Monitor del servidor de aplicaciones de forma dinámica y sin reiniciar, permitiendo iniciar el sistema de trazas sin parar el servicio, mediante las siguientes opciones de la ruta **Traces | VM Traces**:

- **Set LDB Debug Level:** modifica dinámicamente el valor de SystemDebugEnabled.
- **Set Debug Detail Level:** modifica dinámicamente el valor de SystemDebugDetailLevel.

Importante: el hecho de modificar los valores de las variables desde el monitor de forma dinámica no altera los valores de la configuración, por lo que no se graban. Es decir, los cambios no son persistentes.

Almacenamiento de los archivos de traza

Los archivos de traza se almacenan en el directorio denominado m4debug, dentro del directorio establecido como temporal de la configuración en el parámetro *DebugDir*. Este parámetro se modifica en la sección *CVM.LOG* de la configuración de la aplicación, que se encuentra en:

- **Cliente Windows:** En el nodo *CVM.LOG* del fichero de configuración regmeta4.xml. Localice la cadena *DebugDir* en el archivo.
- **Servidor:** En la entrada de registro *DebugDir* que se encuentra bajo la rama *CVM.LOG*. En plataformas Windows en el registro del sistema, en plataformas UNIX en el fichero regmeta4.reg.

```
HKEY_LOCAL_MACHINE\SOFTWARE\Meta4\Mind\3.X\Build\Nombre de la
instancia\APPServer\CVM\LOG
```

Cuando se vuelva a iniciar el Servidor de aplicaciones o el cliente, se utilizarán los nuevos valores introducidos.

Cambio de tamaño

El tamaño por defecto es de 10 MB, pero se puede configurar modificando el valor de la propiedad *MaxLDBInspSize* de la configuración. La unidad empleada para definir el tamaño son los MB.

Este parámetro *MaxLDBInspSize* se modifica en la sección *CVM.LOG* de la configuración de la aplicación, que se encuentra en:

- Cliente Windows: En el nodo *CVM.LOG* del fichero de configuración regmeta4.xml. Localice la cadena *MaxLDBInspSize* en el archivo.
- Servidor: En la entrada de registro *MaxLDBInspSize* que se encuentra bajo la rama *CVM.LOG*. En plataformas Windows en el registro del sistema, en plataformas UNIX en el fichero regmeta4.reg.

HKEY_LOCAL_MACHINE\SOFTWARE\Meta4\Mind\3.X\Build\Nombre de la instancia\APPServer\CVM\LOG

Cuando se vuelva a iniciar el Servidor de aplicaciones o el cliente, se utilizarán los nuevos valores introducidos.

Cambio de número de copias de seguridad

Con la propiedad *MultipleFilesTrace* del archivo regmeta4.reg puede definir si quiere que las trazas de todo el servidor vayan a un par de archivos prefijados o, por el contrario, que cada conexión tenga su propio par de archivos de depuración.

Este parámetro *MultipleFilesTrace* se modifica en la sección *CVM.LOG* de la configuración de la aplicación, que se encuentra en:

- Cliente Windows: En el nodo *CVM.LOG* del fichero de configuración regmeta4.xml. Localice la cadena *MultipleFilesTrace* en el archivo.
- Servidor: En la entrada de registro *MultipleFilesTrace* que se encuentra bajo la rama *CVM.LOG*. En plataformas Windows en el registro del sistema, en plataformas UNIX en el fichero regmeta4.reg.

HKEY_LOCAL_MACHINE\SOFTWARE\Meta4\Mind\3.X\Build\Nombre de la instancia\APPServer\CVM\LOG

Cuando se vuelva a iniciar el Servidor de aplicaciones o el cliente, se utilizarán los nuevos valores introducidos.

Cambio de severidad de errores

Existen cuatro niveles de severidad de errores que se especifican en la propiedad **SystemDebugEnabled** del archivo regmeta4.reg de cada configuración de servidor. Esta propiedad tiene alguno de los siguientes valores:

- 0, sin traza
- 1, traza de datos (sentencias de usuario)
- 2, traza de datos y metadatos (sentencias de usuario y sistema)
- 3, traza de datos, metadatos y funciones de LDB

Este parámetro **SystemDebugEnabled** se modifica en la sección *CVM.LOG* de la configuración de la aplicación, que se encuentra en:

- Cliente Windows: En el nodo *CVM.LOG* del fichero de configuración regmeta4.xml. Localice la cadena **SystemDebugEnabled** en el archivo.
- Servidor: En la entrada de registro **SystemDebugEnabled** que se encuentra bajo la rama *CVM.LOG*. En plataformas Windows en el registro del sistema, en plataformas UNIX en el fichero regmeta4.reg.

HKEY_LOCAL_MACHINE\SOFTWARE\Meta4\Mind\3.X\Build\Nombre de la instancia\APPServer\CVM\LOG

Cuando se vuelva a iniciar el Servidor de aplicaciones o el cliente, se utilizarán los nuevos valores introducidos.

Cambio de nivel de severidad de errores

Existen cinco niveles de detalle que se especifican en la propiedad **SystemDebugDetailLevel** del archivo regmeta4.reg de cada configuración de servidor. Esta propiedad tiene alguno de los siguientes valores:

- 0, no se aplica detalle.
- 1, se muestran detalles relacionados con la ejecución y tiempos.
- 2, se muestran todos los detalles.
- 3, se muestran detalles relacionados con la ejecución, tiempos y datos.
- 4, se muestran detalles relacionados con ejecuciones, tiempos, datos e información de filtros de seguridad.

Este parámetro **SystemDebugDetailLevel** se modifica en la sección *CVM.LOG* de la configuración de la aplicación, que se encuentra en:

- Cliente Windows: En el nodo *CVM.LOG* del fichero de configuración regmeta4.xml. Localice la cadena **SystemDebugDetailLevel** en el archivo.
- Servidor: En la entrada de registro **SystemDebugDetailLevel** que se encuentra bajo la rama *CVM.LOG*. En plataformas Windows en el registro del sistema, en plataformas UNIX en el fichero regmeta4.reg.

HKEY_LOCAL_MACHINE\SOFTWARE\Meta4\Mind\3.X\Build\Nombre de la instancia\APPServer\CVM\LOG

Cuando se vuelva a iniciar el Servidor de aplicaciones o el cliente, se utilizarán los nuevos valores introducidos.

Cambio del formato

No se puede cambiar el formato; la información volcada en este archivo es siempre texto ASCII.

Archivos de log

Estos archivos recogen los errores, avisos (*warnings*) e información de depuración, tanto funcionales como tecnológicos, generados por el servidor de aplicaciones o los clientes. La aplicación genera los siguientes archivos por defecto:

- M4logfile.log
- M4sm_trace.log
- M4wf_trace.log
- M4dmstrace.log
- M4eventlog.log (éste sólo en servidor, consulte el capítulo *Archivos de trazas en el servidor de aplicaciones*)
- M4auditorytrace.log (éste sólo en servidor, consulte el capítulo *Archivos de trazas en el servidor de aplicaciones*)

Todos vuelcan el mismo tipo de información, con diferentes filtros. A efectos prácticos se emplea el m4logfile.log principalmente.

Configuración de los archivos de log

Ubicación

Los archivos de configuración del sistema de mensajes de error se encuentran en el directorio especificado en la entrada **workdir** de la configuración.

- En un puesto cliente (de desarrollo) será el atributo workdir del nodo CLIENT.CVM del fichero de configuración

En dicha ruta existirán los siguientes archivos:

- logbase.ini
- logsys.ini
- logmsg.ini
- logmsg.html

- En un servidor de aplicaciones, será la entrada workdir de la rama:

```
HKEY_LOCAL_MACHINE\SOFTWARE\Meta4\Mind\3.X\Build\Nombre de la
instancia\APPServer\CVM\LOG
```

En dicha ruta existirán los siguientes archivos:

- logbase.ini

- logmsg.ini
- logmsg.html
- logsysserver.ini, a diferencia de logsys.ini el en cliente y además se encontrará en el directorio de configuración del servidor.

En sistemas instalados en más de un idioma puede comprobarse que existen una serie de directorios \Workdir\2, \Workdir\3, etc. Estos directorios contienen los archivos logmsg.ini y logmsg.html traducidos al idioma que corresponde con el código del directorio en que se encuentran.

Sólo para servidores de aplicación

Es posible modificar el nivel de traza desde el monitor del servidor de aplicaciones desde la ruta **Traces | Log**.

- **Disable/Enable Error Messages:** desactiva/activa el volcado de errores.
- **Disable/Enable Warinig Messages:** desactiva/activa el volcado de avisos.
- **Disable/Enable Debug Info Messages:** desactiva/activa el volcado de información de depuración.
- **Disable/Enable Trace Info Messages:** desactiva/activa el volcado de información de traza.

Formato del archivo logbase.ini

El archivo logbase.ini contiene información acerca de los dispositivos de traza que se van a generar (por defecto hay seis, pero se pueden crear más) y de los errores que se van a volcar a cada uno de ellos. Estos se agrupan en módulos:

- [logParameters]: Parámetros globales de las trazas.
- [outputDevices]: Los dispositivos de salida de los mensajes.
- [deviceGroups]: Los grupos de dispositivos que el sistema de trazas utiliza.
- [outputPattern]: Los patrones de salida de severidades.
- [applicationModule]: Los módulos de aplicación al que se asocian los mensajes.

Parámetros globales

Este módulo, encabezado por [logParameters] en el archivo, define el comportamiento general del sistema de recogida de errores.

Las claves que componen esta sección son las siguientes:

- **versión:** Tipo cadena. Especifica la versión de la a partir de la cual el archivo de configuración es valido.
- **showTime:** Tipo booleano. Activa/desactiva la información horaria de cuando se notifico el mensaje.
- **ProcessErrors:** Tipo booleano. Activa/desactiva el filtro global para procesar los mensajes de error.
- **ProcessWarnings:** Tipo booleano. Activa/desactiva el filtro global para procesar los mensajes de aviso.
- **ProcessDebugInfo:** Tipo booleano. Activa/desactiva el filtro global para procesar los mensajes de depuración.
- **ProcessTraceInfo:** Tipo booleano. Activa/desactiva el filtro global para procesar los mensajes de traza.
- **TranslateMessages:** Opciones de traducción. Los posibles valores de esta clave, son los siguientes:
 - TRUE, Habilita la traducción de los mensajes que se vuelcan a los dispositivos de salida (solo se mostrara la parte traducible de los mensajes y la pila completa de llamadas).
 - FALSE, Deshabilita la traducción de los mensajes (se muestra el mensaje sin traducir incluidos los parámetros no traducibles).
 - BOTH, Ambas opciones, se reescribe el mensaje, una traducida y otra sin traducir.
- **numOfOutputDevices:** Tipo. Numérico < 32. Numero de dispositivos de salida.
- **NumOfApplicationModules:** Tipo. Numérico. Numero de módulos de aplicación.

Dispositivos de salida

Este módulo, encabezado por [outputDevices] en el archivo, definen los dispositivos de salida que son los medios de almacenamiento que el sistema utiliza para guardar los mensajes que se notifican. No todos los mensajes se generan se vuelcan al mismo sitio, ni todos los dispositivos de salida son del mismo tipo.

El numero de estos dispositivos esta especificado en la sección anterior y esta limitado a 32, en caso de que este número no coincida con los que realmente están definidos, el sistema de trazas solo reconocerá tantos 'outputdevices' como los que se indiquen en dicha clave.

Todos los dispositivos de salida tienen dos atributos comunes que los identifican:

- **DeviceName:** Tipo cadena. Nombre del dispositivo.

- **DeviceType:** Tipo. Enumerado. Medio de almacenamiento. Los medios de almacenamiento disponibles y sus identificadores son los siguientes:
 - PRIVATE_MESSAGE_QUEUE
 - PBULIC_MESSAGE_QUEUE
 - PUBLIC_LOG_FILE

PRIVATE_MESSAGE_QUEUE

Es una lista privada de mensajes. Ventana reservada en la memoria principal donde se almacenan los mensajes, es de acceso aleatorio y normalmente esta asociado a un 'thread' de ejecución, o a un objeto determinado. De esta forma se pueden separar conjuntos de mensajes e iterar por ellos sin riesgo de mezclarlos. Esta lista es la que se vuelca a pantalla cuando se produce algún error en cliente.

Atributo propio:

- **WindowSize:** Es el tamaño de la ventana de memoria reservada (número máximo de mensajes en la lista) En caso de que se notifique un mensaje y la lista este llena, automáticamente se eliminará el mensaje más antiguo y se añadirá el nuevo. Por defecto se establece su valor a 100.

PUBLIC_MESSAGE_QUEUE

Es una lista pública de mensajes en una ventana reservada en la memoria principal de carácter global, y se trata de una lista compartida por todos los 'threads' y objetos que quieran volcar sus mensajes a este espacio. Al igual que la ventana anterior es posible iterar sobre ella. Los atributos propios de este dispositivo son los mismos que los del anterior.

PUBLIC_LOG_FILE

Son archivos globales. Los parámetros de configuración de este dispositivo son los siguientes:

Atributos propios.

- **filePath:** Tipo cadena. Nombre del archivo de traza.

Grupos de dispositivos de salida

En este módulo, encabezado por [deviceGroups] en el archivo, se definen grupos de dispositivos para que, más adelante, sean utilizados por el sistema de trazas. Estos grupos están limitados a 32 y se definen como máscaras binarias de la siguiente forma:

- **deviceGroupX**: donde X es un entero < 32. Tipo numérico (binario). Es una máscara de bits. Cada posición se corresponde con uno de los dispositivos de salida definidos en la sección OutputDevices. Los dígitos van por orden, es decir, el primero con el primer dispositivo de salida, el segundo con el segundo y así sucesivamente. En la configuración por defecto hay seis dispositivos, por lo que tendremos seis dígitos. Si el valor correspondiente a un dispositivo de salida es 0, éste no pertenecerá al grupo, mientras que si es 1 sí lo hará.

Patrones de salida [outputPattern]

En este módulo, encabezado por [outputPattern] en el archivo, mediante la definición de los patrones de salida, se intenta asociar severidades a grupos de dispositivos. De esta forma, un mensaje procesado, identificado por un código único, puede enviarse o no, a grupos de dispositivos distintos dependiendo de su severidad. El mismo mensaje no tiene por qué tratarse igual si es un error grave o si es un simple aviso.

De esta forma se definen los patrones de salida como:

[outputPatternN]

donde N es un entero < 32

Además con los siguientes atributos:

- **outputPatternName**: Tipo cadena. Nombre del patrón para su identificación.
- **errorDeviceGroup**: Tipo cadena. Nombre del grupo de dispositivos (definido anteriormente) que se utilizara para direccionar los mensajes, en caso de que su severidad sea 'error' y su patrón de salida sea igual a 'outputPatternName'.
- **warningDeviceGroup**: Tipo cadena. Nombre del grupo de dispositivos (definido anteriormente) que se utilizara para direccionar los mensajes, en caso de que su severidad sea 'warning' y su patrón de salida sea igual a 'outputPatternName'.
- **debugDeviceGroup**: Tipo cadena. Nombre del grupo de dispositivos (definido anteriormente) que se utilizara para direccionar los mensajes, en caso de que su severidad sea 'debugInfo' y su patrón de salida sea igual a 'outputPatternName'.
- **traceDeviceGroup**: Tipo cadena. Nombre del grupo de dispositivos (definido anteriormente) que se utilizara para direccionar los mensajes, en caso de que su severidad sea 'traceInfo' y su patrón de salida sea igual a 'outputPatternName'.

Módulos de aplicación [applicationModule]

Finalmente en este módulo, encabezado por [deviceGroups] en el archivo, con la definición de los módulos de aplicación, se asocia un conjunto de mensajes, identificados por un código único a un patrón de salida de los que se han definido anteriormente.

El numero de módulos tiene que ser igual a 'NumOfApplicationModules' especificado en la sección 'logParameters'

La definición de estos módulos de aplicación se hace por secciones de la siguiente forma:

```
[applicationModuleN]
```

donde N es un entero $\leq$ NumOfApplicationModules

Además con los siguientes atributos:

- **modulename**: Tipo cadena. Nombre del módulo para su identificación.
- **initcodenumber**: Tipo numérico. Cota inferior del rango de mensajes que forman este módulo de aplicación.
- **finalcodenumber**: Tipo numérico. Cota superior del rango de mensajes que forman este módulo de aplicación.
- **outputpatternname** (*opcional*): Tipo cadena. Nombre del patrón de salida que va a utilizar este módulo, en caso de no existir esta clave, se asigna al módulo el patrón por defecto 'defaultPattern'.
- **exception** (*opcional*): Con esta clave se identifica un código de mensaje como una excepción. Una excepción asocia un mensaje (perteneciente al módulo en el que se define) a un grupo de dispositivos de salida sin tener en cuenta la severidad con la que se reporta. La semántica para definir una excepción, es la siguiente.

```
exception=codigo_mensaje,nombre_grupo_de_dispositivos
```

donde:

código mensaje: recibe input del finalCodeNumber y pasa output al initCodeNumber

nombre_grupo_de_dispositivos: esta definido en la sección deviceGroups.

Formato de los archivos logsys.ini y logsysserver.ini

Los archivos **logsys.ini** y **logsysserver.ini** contienen la especificación las salidas de los mensajes de error para el cliente y para el servidor respectivamente. Estos archivos tienen una estructura similar a la del **logbase.ini**, aunque simplificada. Normalmente no se necesita modificar el

archivo **logbase.ini** y basta con modificar el archivo **logsys.ini** o **logsysserver.ini**, según el entorno. Este sería el caso de cambiar la ruta de salida de un archivo de traza, por ejemplo. Debe haber consistencia entre ambos archivos; los cambios realizados en el **logsys.ini** o **logsysserver.ini** sobrescriben a los parámetros indicados en **logbase.ini**.

Cambio de severidad de errores

El archivo **logbase.ini** es el encargado de establecer una configuración por defecto para el sistema de trazas, mientras que **logsysserver.ini** o **logsys.ini** parametriza esta configuración para adaptarla a necesidades concretas y que prevalezca sobre la primera.

1. Abra el archivo **logsysserver.ini** con un editor de texto en el servidor o el archivo **logsys.ini** en el cliente.
2. Localice la sección denominada *logParameters*.
3. Modifique las claves *ProcessErrors*, *ProcessWarnings*, *ProcessTraceInfo* y *ProcessDebugInfo*, teniendo en cuenta que su valor es booleano (*True/False*).

Establecimiento del nombre del dispositivo de salida

Otra de las posibilidades que ofrece el archivo **logsysserver.ini** es la de establecer el nombre que se va a emplear para definir un dispositivo de salida (*deviceType*).

Para establecer dicho nombre, siga estos pasos:

1. Abra el archivo **logsysserver.ini** con un editor de texto en el servidor o el archivo **logsys.ini** en el cliente.
2. Localice el parámetro *deviceName*.
3. Entre comillas, indique el nombre que desea asignar a ese dispositivo de salida.
4. Guarde los cambios y salga del editor de texto.

Definición del tipo de dispositivo

Otra de las posibilidades que ofrece el archivo **logsysserver.ini** es la de establecer el tipo para la información de traza que se generará.

Para ello dispone de las opciones:

- **PUBLIC_LOG_FILE:** mensajes de carácter global que permiten contener información de traza. Dentro de éstos se puede especificar el formato con *fileFormat*.
- **PRIVATE_MESSAGE_QUEUE:** mensajes volcados a un espacio de memoria privado, asociado a un hilo de ejecución o a un objeto concreto, que se emplea para pasar los mensajes entre el servidor y el cliente.
- **PUBLIC_MESSAGE_QUEUE:** mensajes de carácter global, volcados a un espacio de memoria pública que contiene las incidencias producidas.
- **NT_EVENT_LOG:** mensajes volcados al gestor de eventos de NT (visibles a través del Visor de eventos).
- **PUBLIC_TCP:** mensajes volcados a una dirección IP prefijada por un puerto determinado, con un cliente en el otro extremo para recuperar dichas trazas.

Para establecer dicho nombre, siga estos pasos:

1. Abra el archivo **logsysserver.ini** con un editor de texto en el servidor o el archivo **logsys.ini** en el cliente.
2. Localice el parámetro *deviceType*.
3. Entre comillas, indique el tipo que desea emplear en ese dispositivo de salida.
4. Guarde los cambios y salga del editor de texto.

Cambio del formato de salida

La información de traza del archivo puede guardarse en varios formatos:

- Texto: formato de texto ASCII.
- Traza: información simple de la incidencia.
- HTML: para verla con un navegador Web.
- XML: contiene la información de los datos y de los metadatos.
- Excel: para ser consultada con la hoja de cálculo.

Para establecer el formato de salida, es necesario configurar el archivo **logsysserver.ini** o **logsys.ini**; para ello, siga estos pasos:

1. Abra el archivo **logsysserver.ini** con un editor de texto en el servidor o el archivo **logsys.ini** en el cliente.
2. Localice la sección *outputDevices* que desee configurar.
3. Busque la clave *fileFormat* e introduzca la opción que desee como formato de salida:
 - TEXT_FORMAT: texto

- TRACE_FORMAT: traza
 - HTML_FORMAT: HTML
 - XML_FORMAT: XML
 - EXCEL_FORMAT: Excel
4. Guarde los cambios y salga del editor de texto.

Elección del idioma del archivo de traza

Otra de las posibilidades que ofrece el archivo **logsysserver.ini** o **logsys.ini** es la de establecer el idioma para la generación de los mensajes de incidencia en el archivo de traza. Existen dos claves que controlan este apartado:

1. Abra el archivo **logsysserver.ini** con un editor de texto en el servidor o el archivo **logsys.ini** en el cliente.
2. Localice [logParameters].
3. Modifique los siguientes claves, según necesite:
 - *translateMessage*: se encarga de la traducción de mensajes en el servidor a un idioma prefijado. Los valores que puede tomar son:
 - *True*: habilita la traducción.
 - *False*: no se realiza traducción.
 - *Both*: repite el mensaje, una vez traducido y otra sin traducir.
 - *language*: establece el valor del idioma al que se traducirán los mensajes. Si está activada la traducción pero no se establece un idioma de traducción, se tomará el inglés por defecto. Los posibles idiomas son:
 - 2, inglés
 - 3, español
 - 4, francés
 - 5, portugués
 - 6, portugués (Brasil)
 - 7, italiano

Códigos de error en los archivos de log

Los errores se codifican concatenando su número de módulo, submódulo y código de error. Sin embargo en el **m4logfile.log**, el código de este error se refleja de otra manera. Para calcular este otro código de error, se han de seguir estos pasos:

1. Pasar el número de módulo a hexadecimal, debiendo tener el número resultante 2 dígitos.
2. Pasar el número de submódulo a hexadecimal, debiendo tener el número resultante 2 dígitos.
3. Pasar el número de error a hexadecimal, debiendo tener el número resultante 4 dígitos.
4. Concatenar los 3 números resultantes.
5. Convertir el número concatenado a decimal. Este número es el que nos aparecerá en el m4logfile cuando se produzca ese error.

EJEMPLO

```
setlog(1,15,16964)
```

Transformamos el uno en hexadecimal = 1 con 2 dígitos 01

Transformamos el 15 en hexadecimal = 0F con 2 dígitos 0F

Transformamos el 16964 en hexadecimal = 4244 con 4 dígitos 4244

quedaría: 01- 0F- 4244

Transformamos todo el número en decimal 010F4244 ----> 17777220.

Así pues, en caso de encontrar un error en el m4logfile y tener necesidad de identificar el módulo, submódulo y error deberemos proceder de manera inversa.

El archivo M4oltp.log

Se trata de un archivo de texto generado por el servidor que muestra el Meta4 Object, el nodo y el método correspondiente a cada transacción OLTP. Incluye el mismo TickCount del archivo m4benchmark.log (tiempo en milisegundos desde el arranque de la máquina) y el identificador del thread.

EJEMPLO

2420823765	2731	SSC_SEC_OPT	SSC_SEC_OPT	IS_ENABLED
2420825343	2732	SRP_TR1_REPORTS	SRP_TR1_REPORTS	LOAD_PRG
2420827796	2735	SSC_SEC_OPT	SSC_SEC_OPT	ROOTLOAD
2420828125	2737	SSC_SEC_OPT	SSC_SEC_OPT	IS_ENABLED
2420830328	2738	SPT_TR_COMPARE	SPT_TR_COMPARE	ROOTLOAD
2420830750	2739	SPT_TR_COMPARE	SPT_TR_COMPARE	ROOTLOAD
2420830968	2740	SPT_TR_COMPARE	SPT_TR_COMPARE	ROOTLOAD

En caso de parada y arranque del servidor de aplicaciones, se incluye una línea que lo indica:

EJEMPLO

```
----- starting dump -----
2359528515 2168 SCH_SESSION ROOT_SESSION REFRESH
2359528703 2284 SCH_SESSION ROOT_SESSION USR_INF
2359528750 2444 SCH_SESSION ROOT_SESSION CONFIRM
2359529234 2402 SAV_PARAMS SAV_PARAMS ROOTLOAD
```

Puede consultar más información sobre la activación, ubicación y configuración de esta traza en el apartado "Configuración común a múltiples trazas" del capítulo "Introducción a las trazas".

El archivo M4metadata.log

Se trata de un archivo de texto generado por el servidor que muestra, para cada transacción de metadatos, el tipo (Meta4Object, presentación o seguridad) y el Meta4Object al que referencia.

Una transacción del tipo M4_SERVICE_METADATA_GET_UPDATE_LIST corresponde a una conexión. El cliente la envía para obtener los objetos en caché que han sido modificados.

Incluye el mismo TickCount del archivo m4benchmark.log (tiempo en milisegundos desde el arranque de la máquina) y el identificador del thread:

EJEMPLO

```
1822894046 1165 M4_SERVICE_METADATA_GET_UPDATE_LIST ---
1822895687 1234 M4_SERVICE_METADATA_CMCR M4_ERRORS
1822921828 943 M4_SERVICE_METADATA_CMCR CSP_TR_TIPO_SS
1822922703 1165 M4_SERVICE_METADATA_CMCR CSP_TR_ESTADO_SS
1822923468 1234 M4_SERVICE_METADATA_CMCR STD_TR_LU_DEP_TYPE
1822971265 943 M4_SERVICE_METADATA_GET_UPDATE_LIST ---
1823003312 943 M4_SERVICE_METADATA_PRESENTATION QBF.CSP_TR_TIPO_SS
```

En caso de parada y arranque del servidor de aplicaciones, se incluye una línea que lo indica y las etiquetas de las distintas columnas:

EJEMPLO

```
----- starting dump for metadata -----
tickCount threadId ChannelType ChannelId
1819508015 943 M4_SERVICE_METADATA_GET_UPDATE_LIST ---
1819943218 1165 M4_SERVICE_METADATA_GET_UPDATE_LIST ---
```

Puede consultar más información sobre la activación, ubicación y configuración de esta traza en el apartado "Configuración común a múltiples trazas" del capítulo "Introducción a las trazas".

Los archivos logon.log y params.log

Estos dos archivos registran el proceso de logon de usuarios, incluyendo los valores leídos del Meta4 Object SCH_SESSION y del de parámetros de aplicación. Se incluye la autenticación del usuario, rol y MSR, además de la validación de sociedad. Por defecto estas trazas están desactivadas.

Para activar estas trazas, localice en la configuración siguiente el parámetro **LogonTrace**, en la sección **Machine** de la configuración de la aplicación, que se encuentra en:

- Cliente Windows: En el nodo **Machine** del fichero de configuración regmeta4.xml. Localice la cadena **LogonTrace** en el archivo.
- Servidor: En la entrada de registro **LogonTrace** que se encuentra en:

`HKEY_LOCAL_MACHINE\SOFTWARE\Meta4\Mind\3.X\Machine`

Realmente, en esta ruta hay dos entradas relevantes a estas trazas:

- **LogonTrace**: permite activar o desactivar estas trazas. Establezca el valor:
 - 1: activa las trazas.
 - 0: desactiva la traza, es el valor por defecto.
- **LogonPath**: los archivos se generan en la ruta indicada por este parámetro de la configuración, el valor por defecto dirige al directorio "\temp" relativa al directorio de configuración de la máquina que corresponde.

Estas entradas sólo se tendrán en cuenta durante el arranque del servidor, por lo que si se cambian durante su proceso de ejecución no se tendrán en cuenta hasta que sea reiniciado.

El archivo LOGON.log contiene las trazas asociadas con el Meta4Object sesión (SCH_SESSION) y se sobrescribe cada vez que el servidor es reiniciado. El archivo PARAMS.log contiene las trazas asociadas a los parámetros de aplicación (Meta4 Object SAV_PARAMS) y no se sobrescribe, cada vez que se reinicia el servidor.

Trazas del motor de informes

El motor de informes dispone de la opción de configurar el volcado de una traza de la actividad realizada al generar un informe. Dicha traza muestra únicamente las operaciones del motor de informes, necesitando normalmente la traza de base de datos lógica para estudiar la carga de datos. Con el fin de obtener un archivo de traza hacen falta dos operaciones:

Configuración de las trazas de informes

Para activar las trazas de informes, localice la entrada **SystemDebugEnabled** en la rama REPORTS de la configuración, que se encuentra en:

- Cliente Windows: En el nodo *REPORTS* del fichero de configuración regmeta4.xml. Localice la cadena **SystemDebugEnabled** en el archivo.
- Servidor: En la entrada de registro **SystemDebugEnabled** que se encuentra bajo la rama *REPORTS*. En plataformas Windows en el registro del sistema, en plataformas UNIX en el fichero regmeta4.reg.

```
HKEY_LOCAL_MACHINE\SOFTWARE\Meta4\Mind\3.X\Build\Nombre de la  
instancia\APPServer\TOOLS\REPORTS
```

Para activar las trazas cambie el valor de la entrada **SystemDebugEnabled** a 1.

Para desactivar las trazas, cambie el valor de la entrada **SystemDebugEnabled** a 0.

Importante: no confundir esta entrada con la del mismo nombre en otra ruta

La activación de estas trazas no requiere el reinicio del servidor de aplicaciones ni de la estación cliente, ya que se lee el valor del registro cada vez que el motor de informes comienza a funcionar.

Para mayor información acerca de la activación de trazas de informes consulte el *Apéndice: Depuración en informes*.

Filtro del nivel de traza

El motor de informes generará la traza con el nivel de detalle que se especifique en las propiedades avanzadas del informe. Es decir, es necesario

modificar las propiedades avanzadas del informe, bien desde el diseñador, bien en la llamada a M4Throw para indicar el nivel de detalle de traza. Si no se indicase no se generaría archivo alguno.

Para detalles sobre los parámetros, consultar las ayudas en línea y el *Apéndice: Depuración en informes*.

Trazas de tramos

Hoy en día existe una manera fácil de trazar las operaciones con tramos.

Para activar las trazas de tramos, localice el parámetro **SystemDebugLevel** en la rama de configuración `CVM.LOG`. Para tener trazas de tramos, **SystemDebugLevel**: debe ser un valor mayor o igual a 3.

Este parámetro **SystemDebugLevel** se modifica en la sección `CVM.LOG` de la configuración de la aplicación, que se encuentra en:

- Cliente Windows: En el nodo `CVM.LOG` del fichero de configuración `regmeta4.xml`. Localice la cadena **SystemDebugLevel** en el archivo.
- Servidor: En la entrada de registro **SystemDebugLevel** que se encuentra bajo la rama `CVM.LOG`. En plataformas Windows en el registro del sistema, en plataformas UNIX en el fichero `regmeta4.reg`.

HKEY_LOCAL_MACHINE\SOFTWARE\Meta4\Mind\3.X\Build\Nombre de la instancia\APPServer\CVM\LOG

Esto genera un archivo llamado **slices.log** en el directorio debug, que viene dado por el parámetro de configuración `DebugDir`.

Las acciones que se trazan son:

- Comienzo de la generación de tramos para un concepto
- Cada tramo que se genera para un concepto
- Cada vez que se crea un tramo por asignación

EJEMPLO DEL CONTENIDO DEL ARCHIVO

```
...
@ Id(272): Generating sliced execution states for item
<SFR_DP_PAYROLL_CHANNEL!SCO_HT_RUN_PAYS.SCO_DELETE_RUN_PAY>|
@ Id(272): Creating slice {1800-01-01 00:00:00} {4000-01-01 00:00:00}
for execution of item
<SFR_DP_PAYROLL_CHANNEL!SCO_HT_RUN_PAYS.SCO_DELETE_RUN_PAY>|
@ Id(272): Creating slice {1800-01-01 00:00:00} {4000-01-01 00:00:00}
to assign value of item
<SFR_DP_PAYROLL_CHANNEL!SCO_HT_RUN_PAYS.SCO_SQL_RUN_PAY> from item
<SFR_DP_PAYROLL_CHANNEL!SCO_HT_RUN_PAYS.SCO_DELETE_RUN_PAY>|
@ Id(272): Generating sliced execution states for item
<SFR_DP_PAYROLL_CHANNEL!SCO_HR_PERIOD.SCO_DELETE_PERIOD>|
@ Id(272): Creating slice {1800-01-01 00:00:00} {4000-01-01 00:00:00}
for execution of item
<SFR_DP_PAYROLL_CHANNEL!SCO_HR_PERIOD.SCO_DELETE_PERIOD>|
...
```


Trazas de Meta4Objects del tipo MT

En este tipo de archivo (MT) el sistema concatena los Meta4Objects que se van llamando y ejecutando en una sesión y agrega todas las propiedades de cada una de los Meta4Object junto con sus valores.

Para activar las trazas de Meta4Objects del tipo MT, localice el parámetro **SystemDebugLevel** en la rama de configuración `CVM.LOG`. Para tener trazas de MT, **SystemDebugLevel**: debe ser un valor mayor o igual a 2.

Este parámetro **SystemDebugLevel** se modifica en la sección `CVM.LOG` de la configuración de la aplicación, que se encuentra en:

- Cliente Windows: En el nodo `CVM.LOG` del fichero de configuración `regmeta4.xml`. Localice la cadena **SystemDebugLevel** en el archivo.
- Servidor: En la entrada de registro **SystemDebugLevel** que se encuentra bajo la rama `CVM.LOG`. En plataformas Windows en el registro del sistema, en plataformas UNIX en el fichero `regmeta4.reg`.

`HKEY_LOCAL_MACHINE\SOFTWARE\Meta4\Mind\3.X\Build\Nombre de la instancia\APPServer\CVM\LOG`

Hay tres tipos de trazas MT identificados por un sufijo & seguido por un ordinal:

- `m4oidentificadormt&1`: corresponde a una vista cliente del Meta4Object
- `m4oidentificadormt&2`: corresponde a una vista servidor del Meta4Object
- `m4oidentificadormt&3`: corresponde a un Meta4Object no separable

Donde `m4oidentificadormt` corresponde al nombre del Meta4Object fuente de la traza.

Estas trazas se vuelcan justo donde se construyen.

El archivo ChannelManager.txt

Al cambiar el valor en **SytemDebugLevel** a 2 se activan otras trazas, por ejemplo el archivo ChannelManager.txt.

En este archivo el sistema concatena los Meta4Objetos que se van llamando y ejecutando en una sesión y agrega todas las propiedades de cada una de los Meta4Object junto con sus valores.

```
Properties = <Size: 22
AUTOLOAD[60] (n) 0}
CURRENCY_PRECISION[62] (n) -1}
DATA_CORRECTION_DATE[190] (d) 2005-02-24 18:12:25
DATA_END_DATE[190] (d) 4000-01-01 00:00:00}
DATA_LANGUAGE[150] (n) 2}
DATA_START_DATE[190] (d) 1800-01-01 00:00:00}
EXECUTE_REAQLSQL_MODE[150] (n) 0}
EXECUTION_DATE[190] (d) 2005-02-24 18:12:25}
EXECUTION_END_DATE[190] (d) 4000-01-01 00:00:00}
EXECUTION_START_DATE[190] (d) 1800-01-01 00:00:00}
ID_APP_ROLE[150] (sv) M4ADM}
iD_ORGANIZATION[190] <NULL>} I
iD_ORGANIZATION_TYPE[22] (n) 0}
iD_RSM[150] (sv) M4ADM}
META_DATA_CORRECTION_DATE[150] (d) 2005-02-24 18:12:25}
META_DATA_END_DATE[150] (d) 4000-01-01 00:00:00}
META_DATA_START_DATE[150] (d) 1800-01-01 00:00:00}
META_LANGUAGE[148] (n) 2}
NUMBER_PRECISION[62] (n) -1}
ROUND_CURRENCY[190] (n) 0}
ROUND_NUMBER[190] (n) 0}
UNIT_DATE[62] (n) 1}>
```

Trazas de depuración: autocache.log

Para cada Meta4Object cuya propiedad **Tipo de ejecución C/S** sea Delta, habrá una copia del Meta4Object mismo en el caché de objetos. Se puede activar las trazas de depuración de estos objetos guardados en el caché delta.

Para ayudar al dimensionamiento y testeo de la caché delta se ha generado un archivo de traza con la siguiente información:

- **TickCount**: el momento en que se produce la operación
- **Event**: puede tomar los siguientes valores:
 - **GET**. Se recupera un objeto de la caché.
 - **PUT**. Se inserta un objeto en la caché.
 - **DELETE**. Se borra un objeto de la caché.
 - **DUMP_BEFORE**. Se va a serializar parte de la caché de memoria a disco.
 - **DUMP_AFTER**. Se ha serializado parte de la caché de memoria a disco.
- **ObjectId**: el identificador del objeto sobre el que se realiza la operación. En las operaciones de GET/PUT se muestra el ObjectId completo. En las operaciones de DELETE se muestra sólo la parte relativa a la sesión de usuario y/o instancia de m4object. En las operaciones de DUMP se muestra "-".
- **ObjectSize**: tamaño del objeto en GET/PUT. En DELETE/DUMP se muestra el valor -1.
- **ObjectsInRAM**: número de objetos en memoria.
- **ObjectsInDisk**: número de objetos en disco.
- **CacheSize**: tamaño en memoria de la caché (en bytes).

Para activar las trazas se utiliza la entrada de registro **SystemDebugLevel** en la rama `CVM\LOG`, donde **SystemDebugLevel** debe tener el valor 2.

Las trazas se vuelcan en un fichero llamado autocache.log que se deja en el directorio especificado en el parámetro **DebugDir** de la misma rama.

Cuando se inicia el archivo, el sistema se le renombra con la fecha actual y una vez llegado al límite de tamaño de archivo se hace un backup (tal como el archivo m4benchmark). Los límites para el dimensionamiento del archivo se fijan en los parámetros de la rama `CVM\LOG`, donde:

MaxRollingFileSize: es el tamaño máxima del archivo antes de que se genere un backup. El valor por defecto es 1024.

MaxRollingFileNum: es en número de archivos "5"

EJEMPLO: DE FICHERO DE TRAZA DELTA

TickCount	Event	ObjectId	ObjectSize	ObjectsInRAM	ObjectsInDisk	CacheSize
105917859	DUMP_BEFORE	-	-1	0	0	1091
105917859	DUMP_AFTER	-	-1	0	0	1091
105917859	PUT	0@SSC_APPUSER	1091	1	0	1091
105918015	DELETE	1@10@	-1	0	0	0
105918015	DUMP_BEFORE	-	-1	0	0	593
105918015	DUMP_AFTER	-	1	0	0	593

Configuración de las cachés

La aplicación utiliza cachés con el fin de mejorar el rendimiento. Los parámetros relativos a cachés se encuentran en la sección *CVM.CACHE* de la configuración de la aplicación, que se encuentra en:

- Cliente Windows: En el nodo *CVM.CACHE* del fichero de configuración regmeta4.xml.
- Servidor: En la rama *CVM.CACHE* del registro. En plataformas Windows en el registro del sistema, en plataformas UNIX en el fichero regmeta4.reg.

```
HKEY_LOCAL_MACHINE\SOFTWARE\Meta4\Mind\3.X\Build\Nombre de la
instancia\APPServer\CVM\CACHE
```

Las entradas de configuración disponibles en Caché incluyen:

- **CacheDir**: directorio a partir del cual se guarda la caché y sus objetos temporales.
- **DataCacheMode**: modo de la cache de datos.
 - NONE: Desactivada.
 - READ: Se lee de disco al arrancar pero no se salva (aunque si se escriben objetos temporales).
 - WRITE: No se lee de disco al arrancar pero si se salva.
 - READ_WRITE: Se lee de disco al arrancar y se salva al salir.
 - MEM_ONLY: Caché solo en memoria (aunque si se escriben objetos temporales).
- **DataCacheMaxMemory**: Memoria máxima (RAM) en bytes que ocupa la caché de datos.
- **DataCacheMaxNumObjects**: Máximo número de objetos en la caché de datos.
- **DataCacheRefreshRatio**: Número de accesos a la caché de datos para que se salve la caché y se revisen sus objetos (por temas de caducidad).
- **DataCacheExpiryRatio**: Ratio de caducidad: se aplica dividiendo la caducidad de los datos cacheados por este ratio. El valor 0 es inválido, se le asignaría el valor 1.
- **DataCacheDefaultMaxPeriod**: Caducidad (en días) por defecto para los objetos de la cache de datos. Si el valor es 0, los objetos no tienen límite de tiempo (por defecto) en la cache. Esta propiedad solo es aplicable a Datos, los metadatos nunca tienen límite de tiempo.
- **MDCacheMode**: Modo de la cache de metadatos.
 - NONE: Desactivada.

- READ: Se lee de disco al arrancar pero no se salva (aunque si se escriben objetos temporales).
- WRITE: No se lee de disco al arrancar pero si se salva.
- READ_WRITE: Se lee de disco al arrancar y se salva al salir.
- MEM_ONLY: Caché solo en memoria (aunque si se escriben objetos temporales).
- **MDCacheMaxMemory**: Memoria máxima (RAM) en bytes que ocupa la caché de datos.
- **MDCacheMaxNumObjects**: Máximo número de objetos en la caché de datos.
- **MDCacheRefreshRatio**: Número de accesos a la caché de metadatos para que se salve la caché y se revisen sus objetos (por temas de caducidad).
- **MDCacheExpiryRatio**: Ratio de caducidad: se aplica dividiendo la caducidad de los metadatos cacheados por este ratio.
- **MDCacheDefaultMaxPeriod**: Caducidad (en días) por defecto para los objetos de la cache de metadatos.
- **LDBCachMode**: modo de cache de la base de datos lógica.

Cachés de Metadatos

Sus referencias se guardan en el archivo: "temporal del cliente"\Cache\CMCRCache.cac

Se encuentran en el directorio "temporal del cliente"\Mcr.

Su extensión es .mcr.

Sus entradas de configuración en el registro son:

- MDCacheDefaultMaxPeriod
- MDCacheExpiryRatio
- MDCacheMaxMemory
- MDCacheMaxNumObjects
- MDCacheMode
- MDCacheRefreshRatio

Cachés de presentaciones

Sus referencias se guardan en el archivo: "temporal del cliente"\Cache\PresCache.cac

Se encuentran en el directorio "temporal del cliente"\ Prs.

Su extensión es .prs.

Sus entradas de configuración en el registro son:

- MDCacheDefaultMaxPeriod
- MDCacheExpiryRatio
- MDCacheMaxMemory
- MDCacheMaxNumObjects
- MDCacheMode
- MDCacheRefreshRatio

Cachés de datos

Sus referencias se guardan en el archivo: "temporal del cliente"\ Cache\
DataCache.cac

Se encuentran en el directorio "temporal del cliente"\ Channel

Su extensión es .chn.

Sus entradas de configuración en el registro son:

- DataCacheDefaultMaxPeriod
- DataCacheExpiryRatio
- DataCacheMaxMemory
- DataCacheMaxNumObjects
- DataCacheMode
- DataCacheRefreshRatio

Cachés de seguridad

Sus referencias se guardan en el archivo:

"temporal del cliente"\ Cache\ CSCRCache.cac

Formato de los archivos

El archivo de trazas está compuesto básicamente por tres elementos:

1. Cada vez que se levanta la máquina correspondiente se vuelca una traza con la configuración de cachés que tiene en ese momento el cliente, un ejemplo de esto podría ser:

```
-CreateCaches  
MdMaxMemory_2000000_MdNumObjects_1_MdRefreshratio_5_DataMaxMemory_100  
0000_DataNumObjects_1_DataRefreshratio_52002-04-17 07:32:52
```

2. Operaciones de alto nivel.
3. Operaciones de bajo nivel. Normalmente las operaciones de alto nivel vienen seguidas de sus correspondiente operaciones de bajo nivel.

A continuación se definen algunos ejemplos de operaciones de bajo nivel:

- Persist: Esta es la operación de escritura en disco, es una operación unitaria, es decir, no tiene inicio de escritura y fin, sino que solo tiene una línea de inicio.

EJEMPLO

```
Persist C:\Program Files\Meta4\Distributed Client\M4Temp\M4Cache\  
client\Cache\CMCRCache.cac2002-04-17 07:32:59
```

- DePersist: Operación de lectura del archivo. Esta operación sí puede tener fin de lectura (EndDePersist), de manera que si ha podido realizar la lectura aparecerá el fin de lectura y si no ha podido, porque el archivo ya no exista o por otra circunstancia, no aparecerá la línea. Un ejemplo de lectura satisfactoria es:

EJEMPLO

Un ejemplo de lectura no realizada es:

```
DePersist C:\Program Files\Meta4\Distributed Client\M4Temp\M4Cache\  
client\Cache\CacheVersion.ver2002-04-17 07:34:05  
EndDePersist...2002-04-17 07:34:05
```

- Delete: operación de borrado de archivo. Pasa lo mismo que con el DePersist, que tiene fin de borrado (EndDelete) si ha encontrado el archivo y lo ha podido borrar y sino ha podido borrarlo no tendrá esta línea.

EJEMPLO

Ejemplo de borrado correcto es:

```
Delete C:\Program Files\Meta4\Distributed Client\M4Temp\M4Cache\  
client\MCR\SAV_PARAMS@03@1@000.mcr2002-04-17 07:33:03  
EndDelete ...2002-04-17 07:33:03
```

Ejemplo de borrado no finalizado:

```
Delete      C:\Program Files\Meta4\Distributed
Client\M4Temp\M4Cache\client\MCR\SAV_PARAMS@03@1@000.mcr2002-04-17
07:33:03
```

Operaciones de alto nivel con sus correspondientes operaciones de bajo nivel son:

- **ActivateCaches:** esta operación se realiza al levantar el cliente y está seguida por una secuencia de **DePersist** que leen los archivos .cac que contienen la definición de las cachés. De esta manera el cliente sabe qué archivos tiene en caché.

EJEMPLO DE ESTA SECUENCIAS ES:

```
ActivateCaches...2002-04-17 07:32:59
DePersist C:\Program Files\Meta4\Distributed Client\M4Temp\M4Cache\
client\Cache\CMCRCache.cac2002-04-17 07:32:59
DePersist C:\Program Files\Meta4\Distributed Client\M4Temp\M4Cache\
client\Cache\CSCRCache.cac2002-04-17 07:32:59
DePersist C:\Program Files\Meta4\Distributed Client\M4Temp\M4Cache\
client\Cache\PresCache.cac2002-04-17 07:32:59
DePersist C:\Program Files\Meta4\Distributed Client\M4Temp\M4Cache\
client\Cache\DataCache.cac2002-04-17 07:32:59
```

- **PersistCaches:** es la operación de alto nivel cuya función es escribir a disco todos los archivos .cac y el archivo .ver, para tener actualizada la lista de objetos en caché y su versión. Tiene que estar seguida por una serie de operaciones de **Persist**.
- **GetUpdatedList:** obtiene los objetos de caché que han sido modificadas.
- **ActualizeCachesByUpdatedList:** Actualiza las cachés con la lista de modificadas que ha obtenido en la operación anterior.
- **RemoveObjectByIdAndVersion:** Actualiza físicamente los archivos que han sido actualizados en la lista de cachés.
- **Refresh:** es la operación de alto nivel que desencadena la actualización de las cachés. Lógicamente después de esta operación vendrán el resto de operaciones que componen la actualización y que ya se han descrito con anterioridad. Un ejemplo de la secuencia completa de actualización podría ser:

EJEMPLO

...

```

Refresh      ...2002-04-17 07:34:05
DePersist   C:\Program Files\Meta4\Distributed Client\M4Temp\M4Cache\
client\Cache\CacheVersion.ver2002-04-17 07:34:05
EndDePersist...2002-04-17 07:34:05
GetUpdatedList...2002-04-17 07:34:05
ActualizeCachesByUpdatedList...2002-04-17 07:34:05
PersistCaches152002-04-17 07:34:05
Persist     C:\Program Files\Meta4\Distributed Client\M4Temp\M4Cache\
client\Cache\CMCRCache.cac2002-04-17 07:34:05
Persist     C:\Program Files\Meta4\Distributed Client\M4Temp\M4Cache\
client\Cache\CSCRCache.cac2002-04-17 07:34:05
Persist     C:\Program Files\Meta4\Distributed Client\M4Temp\M4Cache\
client\Cache\PresCache.cac2002-04-17 07:34:05
Persist     C:\Program Files\Meta4\Distributed Client\M4Temp\M4Cache\
client\Cache\DataCache.cac2002-04-17 07:34:05
Persist     C:\Program Files\Meta4\Distributed Client\M4Temp\M4Cache\
client\Cache\CacheVersion.ver2002-04-17 07:34:05
RemoveObjectByIdAndVersion6_SCH_SESSION2002-04-17 07:33:00
....

```

- **CleanCaches:** se encarga de limpiar la cachés de disco, por lo tanto está seguida por una serie de operaciones Delete que borran los archivos.

EJEMPLO

```

CleanCaches 82002-04-17 07:35:11
Delete      C:\Program Files\Meta4\Distributed Client\M4Temp\M4Cache\
client\CHANNEL\MENUS@@03@0@@1@000.chn2002-04-17 07:35:11
EndDelete

```

Trazas del planificador de tareas

Los archivos de traza descritos anteriormente no contemplan la totalidad de la actividad del servidor de aplicaciones; reflejan las peticiones que provienen de los clientes, no la correspondiente al Planificador de tareas que trabaja sin intervención directa del usuario.

Se detallan a continuación las tablas y columnas que contienen los datos relativos al planificador de tareas. La lista no es exhaustiva, se muestran únicamente las columnas necesarias para determinar la actividad realizada en un periodo de tiempo dado.

Tabla	Columna	Contenido
M4RJS_SCHED_TASKS	ID_SCHED_TASK	Identificador de la tarea planificada.
	SCHED_TASK_NAME	Nombre de la tarea planificada.
	SCHED_TASK_DESC	Descripción de la tarea planificada.
M4RJS_TASK_EXE	ID_SCHED_TASK	Identificador de la tarea planificada.
	ID_TASK_EXE	Identificador de la ejecución de la tarea.
	PLANNED_DATETIME	Fecha de la planificación de la tarea, basado en GMT.
	START_DATETIME	Fecha de inicio de la ejecución de la planificación, basado en GMT.
	END_DATETIME	Fecha de fin de la ejecución de la planificación, basado en GMT.
	SERVER_NAME	Servidor que ejecuta la tarea planificada. Cuando varios servidores de aplicación trabajan conectados al mismo usuario de base de datos, cualquiera de ellos puede ejecutar las tareas planificadas.
	SERVICE_NAME	Servicio que ejecuta la tarea. Habitualmente, hay un solo servicio del Planificador de tareas definido para un servidor de aplicaciones.
	ID_SECUSER	Usuario de aplicación bajo el que se ejecuta la tarea.
	STATUS	Estado de ejecución: • 0: Ejecutado. • 1: En espera. • 2: En ejecución. • 3, 10: En ejecución con parada solicitada. • 4: Expirado. • 5, 6, 7 y 11: Interrumpido.

Tabla	Columna	Contenido
M4RJS_ SUBTASK_EXE	ID_SCHED_TASK	Identificador de la tarea planificada.
	ID_TASK_EXE	Identificador de la ejecución de la tarea.
	ID_SUBTASK_EXE	Identificador de la subtarea dentro de la tarea planificada.
	START_DATETIME	Fecha de inicio de la ejecución de la tarea (con respecto a GMT).
	END_DATETIME	Fecha de fin de la ejecución (con respecto a GMT).
	VM_EXIT_FLAG	Código de retorno: ● 0 : Correcto ● -1 : Error

Trazas en modo desarrollo

3

Acerca de este capítulo

En este capítulo se describe únicamente la traza que está disponible en el cliente Windows de desarrollo **CSTrace.txt** para la depuración del comportamiento de los Meta4Objects entre el servidor y cliente.

Además, se describen las trazas disponibles que pueda utilizar un desarrollador para un cliente Web, tanto para la construcción de páginas JSP como para depurar la funcionalidad desarrollada para el entorno cliente Web.

El archivo CSTrace.txt

El archivo **CSTrace.txt** almacena información sobre el volumen de datos que viaja desde un servidor al cliente y viceversa, y sobre el tiempo que emplea cada transacción cliente/servidor. Este archivo, al contrario que el resto de los descritos en el presente documento, se genera en las máquinas cliente y no en el servidor. Por defecto, las trazas cliente/servidor están desactivadas.

Para activar las cliente/servidor, localice el nodo CVM.CLIENTSERVER del fichero de configuración regmeta4.xml, donde:

- **CSTraceEnabled:** permite activar el nivel de traza
 - 0: desactivada. Es el valor por defecto.
 - 1: activa un nivel de traza.
 - 2: activa un nivel superior de traza.
- **CSTraceOutputFileName:** la ruta donde se genera el archivo. Si no se especifica una ruta concreta, el archivo se generará en la ruta del ejecutable M4Mind.exe.
- **CSTraceShowCachedOperations:** indica si se mostrarán las operaciones sobre datos almacenados en caché.
 - 0: no se mostrarán. Es el valor por defecto.
 - 1: se mostrarán.

Nota: se puede simular la actividad cliente/servidor desde una estación de desarrollo activando las entradas de registro. De esta forma no es necesario tener un servidor para realizar pruebas de tráfico de datos. Además, dado que en este caso las partes servidor y cliente están en la misma máquina no se sobrecarga la red.

En el archivo aparecen tabuladas de izquierda a derecha las siguientes columnas:

- **SERVICE:** Cadena de caracteres con el nombre del servicio lógico asignado a la petición (OLTP, METADATA, ...).
- **ROLE:** Rol del usuario que desencadena la transacción.
- **INIT_TIME:** Momento en el que se inicia la transacción.
- **END_TIME:** Momento en que finaliza la transacción.
- **ELAPSED_TIME:** Tiempo transcurrido (en segundos).
- **COMMS_OUT:** No está implementado.
- **COMMS_IN:** No está implementado.

-
- FROM_CACHE: True / False. Por defecto, solo se muestran las operaciones que no están cacheadas.
 - DATA_OUT: Volumen de datos que viaja del cliente al servidor. Está expresado en bytes. No incluye cabeceras de pdu, ni cabeceras de red, ni archivos (campos de tipo Blob). Expresa el volumen de datos después de descomprimir.
 - DATA_IN: Volumen de datos que viaja del servidor al cliente. Está expresado en bytes. No incluye cabeceras de pdu, ni cabeceras de red, ni archivos (campos de tipo Blob). Expresa el volumen de datos después de descomprimir.
 - INFO: Método que desencadena la transacción.

Si se establece un nivel 2, además de la información mencionada aparece información sobre el tamaño de serialización y deserialización.

Trazas de presentaciones

Con la finalidad de contar con una información completa de trazas para analizar el rendimiento de las presentaciones, el diseñador de presentaciones dispone de la opción de generar e imprimir información completa de las acciones realizadas por el usuario.

Adicionalmente al archivo **cs_trace** comentado en el apartado anterior, tiene la posibilidad de generar un archivo de trazas que ofrece información completa por acción de usuario, enlazada con las transacciones realizadas a nivel de mv (Oltp, Metachanel, XML, etc.).

En el archivo **cs_trace** se pueden tener muchas transacciones pero no se sabe cuántas de ellas se corresponden con que el usuario ha pulsado un botón en una determinada presentación. La información que se vuelca en el **cs_trace** se puede configurar con diferentes niveles. Para el nivel uno, se asegura que una línea del archivo es una transacción. Para los niveles restantes, por cada transacción ya no hay una sola línea en el archivo **cs_trace**.

Para volcar la información asociada a las acciones de usuario, se creará un nuevo archivo de trazas en el directorio configurado por el usuario en la entrada de registro **DebugDir**, con un nombre predeterminado: **vbtrace.txt**.

Dentro de este archivo vbtrace.txt, también es posible recoger las trazas de ventanas modales (mensajes de error, avisos de confirmación, filtros QFB, etc.) para calcular los tiempos de retardo en mostrar la ventana hasta que el usuario hace clic en el botón Aceptar.

En el nuevo archivo de trazas, por cada acción de usuario, se tendrá una primera línea describiendo la acción realizada por el usuario y después vendrán "n" líneas, correspondientes a las transacciones asociadas a la acción del usuario.

El formato del nuevo archivo de trazas será similar al **cs_trace**, pero se añadirán unas nuevas columnas que contendrán los valores que describen la acción realizada por el usuario.

Las columnas contenidas en el archivo son las siguientes:

- **SERVICE**: Tipo de línea de traza. Estos son:
 - **FB** (FUNCTION_BEGINS): Comienza la función interna desencadenada por la acción del usuario
 - **FE** (FUNCTION_ENDS): Finaliza la función interna desencadenada por la acción del usuario
 - **AB** (ACTION_BEGINS): Comienza la acción realizada por el usuario
 - **AE** (ACTION_ENDS): Termina la acción realizada por el usuario

- **WMB** (WINDOW_MODAL_BEGINS): Acción que muestra la ventana modal
- **WME** (WINDOW_MODAL_ENDS): Deja de mostrar la ventana modal
- **MMB** (MODAL_MESSAGE_BEGINS): Muestra mensaje modal (error, confirmación, etc.)
- **MME** (MODAL_MESSAGE_ENDS): Deja de mostrarse el mensaje modal
- **OBJECT**: identificador del objeto dentro de la cual se están realizando las distintas acciones de usuario. Por ejemplo, si es dentro de una presentación, sería el identificador de la presentación.
- **FUNCTION**: indica la función de bajo nivel que ha desencadenado la acción de alto nivel de usuario.
- **ACTION**: indica la acción de alto nivel que ha realizado el usuario. Sería el comienzo de un conjunto de funciones, es decir sería el comienzo de un bloque de traza.
- **INI_TIME** y **END_TIME** (Fecha de Inicio y de Fin de la Función/Acción): Indicaría la fecha de inicio y de fin incluyendo las milésimas de segundo de las funciones y acciones de usuario trazadas.
- **ELAPSED**: Muestra la diferencia entre el INI_TIME y el END_TIME, es decir, la duración de la función/acción.

Configuración del archivo de trazas

La configuración del nuevo archivo de trazas de las acciones de usuario o trazas VB se realiza modificando el parámetro VBTraceEnabled de la configuración de la aplicación, que se encuentra en el nodo CVM.CLIENTSERVER del fichero de configuración regmeta4.xml.

El archivo se llamará **vbtrace.txt** y se generará dentro del directorio de debug que se tenga configurado en la aplicación que es usualmente DebugDir en el nodo CVM.LOG del fichero de configuración regmeta4.xml.

Niveles de trazas

Existen distintos niveles de trazas a ser configurados por el usuario de acuerdo a la información deseada. Los valores por cada nivel son los siguientes:

- Valor "0": No hay traza
- Valor "1": Todas las trazas (Ventanas modales, funciones y acciones)
- Valor "2": Trazas de acciones y ventanas modales
- Valor "3": Trazas de acciones únicamente

- Valor "4": Trazas de funciones internas únicamente.

Trazas de transferencia de datos entre Meta4Objects

Para activar dichas trazas deberá establecer el valor 1 en las entradas de registro **TraceTransferData** de la rama CVM\LOG del archivo de registro de aplicación regmeta4.xml. Por defecto, esta entrada tendrá valor 0, es decir, las trazas no estarán activadas. Si necesita ejecutar en un entorno cliente-servidor, aunque no se recomienda, recuerde que debe actualizar la entrada en las ramas de registro de la parte cliente y de la parte servidora.

Las trazas del proceso de transferencia de datos entre Meta4Objects se van volcando secuencialmente en un mismo archivo "m4TransferData_<user>.txt", donde <user> es el ID Usuario que ejecuta el proceso. Cuando el archivo llegue a su tamaño máximo o bien cuando se inicie una nueva sesión para ese usuario y se vaya a tener que escribir sobre un archivo ya existente, previamente a la inicialización, se copiará el archivo existente "m4TransferData_<user>.txt" como "m4TransferData_<user>_Previous.txt".

El tamaño máximo del archivo se configura en la entrada **MaxLDBInspSize** del archivo regmeta4.xml. Este valor contempla el tamaño de los dos archivos de log (el normal y el previo), por lo que el tamaño de cada uno de ellos será como máximo la mitad del valor indicado en esa entrada.

Cada vez que se genere el archivo "m4TransferData_<user>_Previous.txt", debido al control del tamaño (y no a una nueva sesión), se indicará este hecho tanto en el pie del archivo que se está copiando como en la cabecera del archivo que se está iniciando con el texto "File swapped".

La configuración de la ubicación del archivo de traza se especifica en la entrada **DebugDir** del archivo regmeta4.xml.

El uso recomendado del sistema de traza es en un cliente de desarrollo, no en un cliente Rich Web.

Trazas del cliente ligero

Existen varias trazas disponibles para el cliente Web o el cliente ligero.

Trazas de compilación e instanciación de páginas JSP

Las trazas de las compilaciones de páginas JSP y las instanciaciones de clases e inicializaciones de las páginas JSP que se ha vuelto a modificar.

Las trazas de las compilaciones JSP, por defecto se encuentran en:

```
C:\Program Files\meta4\M4wss\defaults\m4jspc\log
```

Las trazas de las instanciaciones de clases se encuentran en:

```
C:\Program Files\meta4\M4wss\defaults\m4jpsi\log
```

Consulte el capítulo Compilación e instanciación de páginas JSP en el manual de *HTML Development Framework*.

Trazas necesarias en la depuración de páginas JSP

Cuando se depura una página JSP en el cliente Web, hay que consultar varios tipos de trazas en distintos puntos: entre el cliente, el servidor web y el servidor de aplicaciones.

Consulte el capítulo Compilación e instanciación de páginas JSP en el manual de *HTML Development Framework*.

Trazas generados mediante las funciones Log de LN4

Una vez que se haya activado el nivel de depuración mediante `SystemDebugEnabled` y `SystemDebugLevel`, puede utilizar el lenguaje LN4 para el tratamiento y la generación de logs, por ejemplo, utilizar la función `SetLog`. Estas funciones permiten generar mensajes de error que se visualizarían o emitirían en función del nivel de depuración establecido en `SystemDebugEnabled` y `SystemDebugLevel`.

Consulte el capítulo Gestión de errores en el manual de *HTML Development Framework*.

Consulte también las funciones LN4 en las ayudas en línea.

Apéndice: Trazas de cliente servidor

4

Introducción

A continuación se muestran ejemplos de dos niveles de trazas de archivos cliente servidor.

Ejemplos de Trazas

Ejemplo de archivo a nivel 1

2004/07/02 10:31:12 M4ObjectEngine Client Started											
2004/07/02 10:31:20 User Session Started, Server=localhost, User=M4ADM											
TAG	SERVICE	ROLE	INIT_TIME	END_TIME	ELAPSED	COMMS_OUT	COMMS_IN	FROM_CACHE	DATA_OUT	DATA_IN	INFO
MetaChannel			10:31:21	10:31:21	0.000	0.000	0.000	TRUE	0	0	SSC_APPUSER, Front
MetaChannel			10:31:22	10:31:22	0.000	0.000	0.000	TRUE	0	0	SCH_SESSION, Both
MetaChannel			10:31:22	10:31:22	0.000	0.000	0.000	TRUE	0	0	ELIGIBILITY_CHANNEL, Front
MetaChannel	M4ADM		10:31:23	10:31:23	0.000	0.000	0.000	TRUE	0	0	SAV_PARAMS, Front
OLTP		M4ADM	10:31:23	10:31:23 0.032	0.000	0.000	0.000	FALSE	806	1036	SAV_PARAMS!CONCEPTS.LOAD_PRG() Object List: SAV_PARAMS (Front)
OLTP		M4ADM	10:31:23	10:31:23 0.218	0.000	0.000	0.000	FALSE	1915	3139	SAV_PARAMS!RET_VALUES_LONG.LOAD_PRG() Object List: SAV_PARAMS (Front)
MetaChannel	M4ADM		10:31:23 0.000	10:31:23 0.000	0.000	0.000	0.000	TRUE	0	0	SCH_SESSION, Both
MetaChannel	M4ADM		10:31:23 0.000	10:31:23 0.000	0.000	0.000	0.000	TRUE	0	0	SAV_PARAMS, Front
OLTP		M4ADM	10:31:24	10:31:24 0.000	0.000	0.000	0.000	FALSE	768	642	SAV_PARAMS!UTILS.ROOTLOAD() Object List: SAV_PARAMS (Front)

Ejemplo de archivo a nivel 2

```

2004/07/02 10:31:12 M4ObjectEngine Client Started
2004/07/02 10:31:20 User Session Started, Server=localhost, User=M4ADM

TAG SERVICE ROLE   INIT_TIME END_TIME  ELAPSED  COMMS_  COMMS_  FROM_  DATA_  DATA_  INFO
                OUT      IN      CACHE OUT      IN
MetaChannel        10:31:21 10:31:21  0.000    0.000   0.000  TRUE   0       0       SSC_APPUSER, Front
MetaChannel        10:31:22 10:31:22  0.000    0.000   0.000  TRUE   0       0       SCH_SESSION,Both
MetaChannel        10:31:22 10:31:22  0.000    0.000   0.000  TRUE   0       0       ELIGIBILITY_CHANNEL,
Front
MetaChannel M4ADM  10:31:23 10:31:23  0.000    0.000   0.000  TRUE   0       0       SAV_PARAMS, Front
OLTP              M4ADM  10:31:23 10:31:23  0.032    0.000   0.000  FALSE  806    1036   SAV_PARAMS!CONCEPTS.
LOAD_PRG() Object
List:SAV_PARAMS
(Front)

        Id = SERIALIZE  Size = 691  Child count = 1
        Id = SAV_PARAMS  Size = 108
        Id = DESERIALIZE Size = 924  Child count = 1
        Id = SAV_PARAMS  Size = 364

OLTP              M4ADM  10:31:23 10:31:23  0.218    0.000   0.000  FALSE 1915   3139   SAV_PARAMS!RET VALUE
S_LONG.LOAD_PRG()
Object
List:SAV_PARAMS
(Front)

        Id = SERIALIZE  Size = 1800  Child count = 1
        Id = SAV_PARAMS  Size = 1060

```


Apéndice: Volcado de un rango de errores a un nuevo archivo

5

Introducción

Según lo indicado será necesario modificar los archivos logbase.ini y logsys.ini o logsysserver.ini, dependiendo de si estamos trabajando en un puesto cliente o en un servidor.

Se adjuntan dos ejemplos de logsys.ini y logbase.ini que se tomarán como punto de partida de este ejemplo. Se pretende desviar un rango de errores a un archivo específico.

Modificaciones en el archivo logbase.ini

Crea un nuevo dispositivo de salida. Para ello, proceda de la siguiente manera:

Modifique la sección: [logParameters]

Incremente "numOfOutputDevices" en una unidad, ya que se creará un archivo de traza nuevo.

Incremente "numOfApplicationModules" en dos unidades, ya que creará dos módulos de aplicación nuevos, y modificar uno.

```
numOfOutputDevices= 7
numOfApplicationModules= 52
```

·Nueva Sección: [outputDevice7]

Creemos esta sección para parametrizar el nuevo archivo de traza. Se pueden crear hasta ocho dispositivos de salida.

```
deviceName="traza_ejemplo"
deviceType="PUBLIC_LOG_FILE"
filePath=" traza_ejemplo.txt"
```

```
fileSize=100000
fileFormat="TRACE_FORMAT"
```

·Sección: [deviceGroup]

Tenemos un nuevo dispositivo de salida. Así pues deberemos añadir un bit más a la máscara de los grupos. Cada grupo de dispositivos (device group) es la referencia interna a la que se va a enviar cada patrón de salida (outputpattern). Para cada grupo de dispositivos se tiene una máscara en la que indicamos si el patrón de salida que utiliza este grupo se envía a uno o más archivos de traza. La máscara se compone de tanto dígitos binarios como patrones de salida tengamos. Así para siete outputdevices necesitamos siete dígitos. Estos dígitos están en orden de forma que el primero se corresponde con [outputDevice1]. Se pueden crear hasta ocho grupos de dispositivos. En nuestro caso modificaremos el séptimo grupo.

"Outputdevice1" se corresponde con la cola de errores local del sistema y es necesario para que el error viaje del servidor al cliente, por lo que no se tocará.

```
[deviceGroup]
deviceGroup1=1100000
deviceGroup2=0010000
deviceGroup3=0001000
deviceGroup4=0100000
deviceGroup5=1000000
deviceGroup6=1100100
deviceGroup7=0000001
deviceGroup8=1100000
```

·Nueva sección: [OutputPattern9]

Esta sección es también nueva. A ella se va a hacer referencia cuando se indique el tratamiento recibe los distintos tipos de errores en función de su nivel de gravedad (error, warning, etc.) En este nuevo patrón de salida indicamos que se vuelquen tanto los errores como los warnings y la información de depuración. Así pues, desviaremos la salida de los errores al devicegroup7. En cuanto a la información de traza, no queremos volcarla así que emplearemos devicegroup0. Como se puede ver éste no ha sido definido como grupo de dispositivos y se emplea para anular el volcado de información de un tipo concreto en un patrón de salida

```
outputPatternName="patron_de_salida_ejemplo"
errorDeviceGroup="deviceGroup7"
warningDeviceGroup="deviceGroup7"
debugDeviceGroup="deviceGroup7"
traceDeviceGroup="deviceGroup0"
```

Secciones: [applicationModule1], [applicationModule56] y

[applicationModule57] (nuevas las dos últimas)

Vamos a modificar la sección para acotar un rango de errores específico y volcarlo en nuestro archivo de ejemplo. Dividiremos el intervalo de errores asignado a la parte funcional de en tres intervalos nuevos. El segundo intervalo será el correspondiente a los errores que queremos desviar al nuevo archivo. Los otros dos intervalos se volcarán como siempre, salvo el último que es una excepción y pretendemos volcarlo también en nuestro archivo de ejemplo. Los números de error están expresados en base decimal. Partimos de la siguiente situación:

```
[applicationModule1]
moduleName="M4_FUNCTIONAL"
initCodeNumber=16777216
finalCodeNumber=4294967295
outputPatternName="debugPattern"
```

Dividiremos los rangos de errores de la siguiente manera:

```
16777216 - 20000000
20000001 - 30000000
30000001 - 4294967295
```

Así pues, modificaremos el módulo 1 y crearemos los módulos 56 y 57

```
[applicationModule1]
moduleName="M4_FUNCTIONAL"
initCodeNumber=16777216
finalCodeNumber=20000000
outputPatternName="debugPattern"

[applicationModule56]
moduleName="M4_FUNCTIONAL_NEW_FILE"
initCodeNumber=20000001
finalCodeNumber=30000000
outputPatternName="patron_de_salida_ejemplo"

[applicationModule57]
moduleName="M4_FUNCTIONAL2"
initCodeNumber=30000001
finalCodeNumber=4294967294
outputPatternName="debugPattern"
exception=4294967295,DeviceGroup7
```

Como vemos, volcamos los rangos primero y último en el patrón "debugpattern"

que está definido en la sección [outputPattern4], que vuelca mensajes de error, warnings y mensajes de depuración en los dispositivos 1 y 2: la cola privada de mensajes (los que aparecen por pantalla) y el m4logfile.log. En cuanto al segundo rango, saldrá a nuestro patrón de ejemplo, que vuelca los errores, warnings y mensajes de depuración en el dispositivo 7, el cual escribe en el archivo traza_ejemplo.txt que hemos creado. También vemos cómo manejamos la excepción del último mensaje y cómo se le indica a qué grupo de dispositivos debe salir.

Modificaciones en el archivo logsys.ini

El archivo logsys.ini tiene la estructura de logbase.ini, pero simplificada. Normalmente no se necesita modificar el archivo logbase.ini y basta con modificar el archivo logsys.ini. Este sería el caso de cambiar la ruta de salida de un archivo de traza, por ejemplo.

Los cambios que tenemos que hacer son para mantener la consistencia entre logbase.ini y logsys.ini. Hay que tener en cuenta que los valores que aquí se pongan en los parámetros sobrescriben los indicados en logbase.ini.

·Sección: [logParameters]

Ya que hemos creado un nuevo dispositivo, debemos modificar la entrada:

```
numOfOutputDevices= 5
```

Hay que hacer notar que el logsys.ini original sólo emplea cuatro dispositivos a diferencia del logabase.ini, que tiene definidos seis.

·Nueva sección: [outputDevice5]

Creamos esta sección para parametrizar el nuevo archivo de traza. El nombre que debemos dar al dispositivo es el mismo que en el logbase.ini. El nombre de la sección no importa que no corresponda en logbase.ini al mismo número de outputdevice; lo que sí es importante y se utiliza para la redefinición, es el valor de "deviceName". Se sobrescribirá por tanto los valores cuyo "deviceName" es "traza_ejemplo". Obsérvese que en "filePath" se ha puesto la ruta completa al archivo y no solamente el nombre del mismo.

```
[outputDevice5]
deviceName="Nuevo archivo traza"
deviceType="PUBLIC_LOG_FILE"
filePath="c:\Archivos de Programa\M4temp\traza_ejemplo.txt"
fileSize=100000
fileFormat="TRACE_FORMAT"
```

Apéndice: Depuración en informes

6

Introducción

En este apéndice se explica como activar las trazas de los informes. Para depurar informes, primero hay que activar las trazas de los informes en el sistema. Después puede utilizar el comando DEBUG y sus parámetros para generar las trazas de depuración.

Activación y desactivación de trazas de informe

Al activar las trazas de informes se generará una traza por informe ejecutado. En cada sesión de aplicación, el sistema comprobará

El nivel de traza de informes se define en el archivo **regmeta4.reg**, en el apartado "**SystemDebugEnabled**" del fichero de configuración regmeta4.xml.

Modifique **SystemDebugEnabled** e introduzca:

- 1: se activa el sistema de trazas de informes.
- 0: se desactiva el sistema de trazas de informes.

4. (Opcional) Modifique **TraceFile** e introduzca la ruta y nombre del archivo de traza.

Este nombre del archivo funcionaría como un prefijo. Al generar trazas del motor de informes en el sistema, se utilizar la siguiente nomenclatura:

Prefijo_N_ENG.txt

Donde

- Prefijo: es el valor en TraceFile.
- N: representa un número n que incrementará, n+1, con cada ejecución nueva.
- ENG: sufijo que indica que el tipo de traza corresponde a la ejecución del informe desde el motor de informes.

Definición del nivel de depuración

En Informes puede depurar los informes en tiempo de ejecución y también las consultas desde la herramienta de consultas.

Para usar el grupo indicado en modo depuración, establezca el nivel de detalle en modo depuración.

A continuación puede depurarse los informes y su código utilizando el comando DEBUG y sus parámetros.

Dónde se puede usar este comando:

En la opción de menú Informes

1. Seleccione la opción de menú **Herramientas de pruebas de informes | Ejecutar informes**. Si el informe está definida para la salida predefinida, aparecerá la pantalla correspondiente.
2. Seleccione la pestaña Opciones de seguimiento para probar y definir informes de ajuste e introducir otros parámetros.
 - Modo prueba gráfico: active esta casilla de verificación para habilitar el modo de prueba gráfico. Esta opción es útil para alinear datos. Sólo es válida para la salida al visor.
 - Modo de depuración: active esta casilla de verificación para habilitar el modo de depuración. Sólo está disponible si ha hecho la instalación en modo de depuración.
 - Otros parámetros: en este cuadro, puede establecer parámetros de ejecución del informe, que se pasarán secuencialmente. Los parámetros varían según el tipo de salida.

En la herramienta de usuario Consulta

1. Seleccione la consulta.
2. Haga clic en la opción de menú **Herramientas | Salidas Predefinidas**.
3. Seleccione una de las salidas predefinidas que tenga establecido o crea una.
4. Ejecútelo, y si el informe está definida para la salida predefinida, aparecerá la pantalla correspondiente.

Comando DEBUG y sintaxis:

La sintaxis de los parámetros es la siguiente:

```
/DEBUG:GroupName: IGNORE/GENERAL/DETAILED/CRAZY
```

Donde

GroupName puede ser cualquiera de los siguientes:

- **APICHN**: este grupo ofrece información sobre cómo acceder a los Meta4Objects.
- **LOADER**: este grupo ofrece información sobre cómo cargar el diseño del informe.
- **LOADERCN**: este grupo ofrece información sobre cómo cargar los datos del informe.
- **CORE**: este grupo ofrece información sobre la gestión de secciones, saltos de página, etc.
- **REPARG**: este grupo ofrece información sobre la gestión de parámetros de entrada.
- **MEMPROF**: este grupo ofrece información sobre la cantidad de memoria dinámica usada.
- **ANALEX**: este grupo ofrece información sobre la corrección de operadores de expresiones.
- **TEMP**: este grupo ofrece información sobre los archivos temporales generados.
- **ALL**: El término reservado ALL hace que el comando active a todos los grupos anteriores.

EJEMPLO

```
/DEBUG:ALL:IGNORE /DEBUG:MEMPROF:CRAZY
```

Este comando desactivaría todos los grupos y después activaría el grupo MEMPROF.

El nivel de detalle de depuración puede ser cualquiera de los siguientes, de menor a mayor niveles de depuración:

- **IGNORE**: no se depura.
- **GENERAL**: corresponde al nivel 1 de depuración.
- **DETAILED**: corresponde a nivel 2 de depuración con más detalles.
- **CRAZY**: corresponde a nivel 3 de depuración con todos los detalles posibles.

Apéndice: Modelo de datos del planificador de tareas y Volcado de trazas

7

Introducción

En este apéndice se suministra los datos que el planificador de tareas ofrece para cada tarea y para cada ejecución de una tarea planificada.

Modelo de datos del planificador de tareas

Definición de los datos suministrados por el modelo

Datos de la planificación

Se modifican cuando el usuario planifica una nueva tarea:

- M4RJS_SCHED_TASKS: Definiciones de tarea planificada.
- M4RJS_SCHED_TASKS1: Definiciones de tarea planificada.
- M4RJS_SCHED_TASKS2: Definiciones de tarea planificada.
- M4RJS_DEF_PARAMS: Parámetros de entrada.
- M4RJS_DEF_PARAMS1: Parámetros de entrada (valor).

Datos de las ejecuciones

Almacenan los datos de las distintas ejecuciones. Se modifican a medida que se ejecutan los grupos de tareas planificados.

- M4RJS_TASK_EXE: Ejecuciones de tarea.
- M4RJS_SUBTASK_EXE: Ejecuciones de subtarea
- M4RJS_SUBTASK_EXE1: Ejecuciones de subtarea (en formato de traza).
- M4RJS_EXE_PARAMS: Parámetros de entrada y de salida de cada ejecución de subtarea.
- M4RJS_EXE_PARAMS1: Parámetros de las ejecuciones (valor).
- M4RJS_EXE_RES: Información sobre los recursos asociados, sin el contenido.
- M4RJS_EXE_RES1: Información sobre los recursos asociados (en formato de una ruta).
- M4RJS_RESOURCES: Contenido de los recursos asociados.
- M4RJS_RESOURCES1: Contenido de los recursos (en formato blob).

Configuración de la instalación

Define los servidores que trabajan sobre el mismo usuario de base de datos. Lo establece el administrador y un usuario normal no debería modificarlo:

- M4RJS_SITE_DEF: Distintos servicios de la instalación

Datos de sistema

Contienen datos fijos y no deben modificarse:

- M4RJS_ZONEINFO_DST: Información sobre las distintas franjas horarias.
- M4RJS_JOB_TYPES: Tipos de trabajo (LN4 o administración; nombres traducidos).
- M4RJS_JOB_TYPES1: Tipos de trabajo (descripciones traducidas).
- M4RJS_EXEC_STATUS: Estados de ejecución (nombres traducidos).
- M4RJS_EXEC_STATUS1: Estados de ejecución (descripciones traducidas)

Volcado de trazas

Para realizar el volcado de trazas usamos la función **UpdateLog**

Es una función que se ofrece para trazar ejecuciones planificadas (JobScheduler).

En la ejecución planificada de métodos largos se pueden ir volcando los warning al fichero de traza a medida que se invoca al UpdateLog. De esta forma se puede saber por dónde va la ejecución del método.

Si no se utiliza el UpdateLog las trazas no se vuelcan hasta el final de la ejecución del método.

¿Cómo invocarlo?

Directamente puede ser un ítem método tipo dll sobre la dll m4jsutils y método UpdateLog, tal y como está definido en el Meta4object de prueba. Este método será al que se invoque desde LN4 cuando se quieran volcar las trazas.